Freie Universität Berlin

Bachelorarbeit am Institut für Informatik der Freien Universität Berlin

Arbeitsgruppe Software Engineering

Electricity Price-Aware HPC Scheduling

Watsche Khalatyan
Matrikelnummer: 5571491
watschek03@zedat.fu-berlin.de
w.khalatyan@gmail.com

Betreuer/in:	Dr.-Ing. Barry Linnert
Eingereicht bei:	Dr.-Ing. Barry Linnert
Zweitgutachter/in:	Prof. Dr.-Ing. Jochen Schiller

Berlin, 26. Oktober 2026

Abstract

Background. Electricity prices on the day-ahead market change every hour, but HPC batch schedulers decide when jobs run without regard to price. **Objective.** This thesis examines whether a scheduler that knows the hourly price and each job's hourly activity profile can reduce the price paid for active computation without unacceptable losses in waiting time and deadline compliance. **Methods.** An hourly simulator of a 64-node cluster, whose code, data, and results are public, compares first-come, first-served (FCFS) scheduling, EASY backfilling, and a price-aware policy, P3R, that pauses and resumes checkpointable jobs, weighing saving against deadline risk with one parameter. All policies run on identical synthetic workloads at four load levels, driven by the 2024 German day-ahead price trace, with paired comparisons over ten seeds. **Results.** At low load P3R pays a price 29 % below EASY; jobs wait about three hours longer for their start, finish 17 hours later on average because of pauses, and the deadline-miss rate rises from 0.6 to 1.4 %. At medium load it pays 13 % less, but the fully price-seeking setting doubles the mean waiting time and raises the miss rate from 16.5 to 26.1 %, ten percentage points above EASY. A moderate setting keeps 42 % of the saving with no later completion and a miss rate below EASY's. Beyond capacity the saving falls below 1 %. Most of the saving comes from shifting work in time; knowing the activity profiles adds 1.3 points at low load. **Conclusions.** Price shifting pays off when the cluster has spare capacity and jobs have slack; under overload it has nothing to work with. The savings are an upper bound under free checkpointing and perfect price knowledge.

Eidesstattliche Erklärung

Ich versichere hiermit an Eides Statt, dass diese Arbeit von niemand anderem als meiner Person verfasst worden ist. Alle verwendeten Hilfsmittel wie Berichte, Bücher, Internetseiten oder ähnliches sind im Literaturverzeichnis angegeben, Zitate aus fremden Arbeiten sind als solche kenntlich gemacht. Die verwendeten Software- und KI-Werkzeuge sowie Art und Umfang ihres Einsatzes sind in Anhang A dokumentiert. Die Arbeit wurde bisher in gleicher oder ähnlicher Form keiner anderen Prüfungskommission vorgelegt und auch nicht veröffentlicht.

Berlin, 26. Oktober 2026

Watsche Khalatyan

Acknowledgments

I would like to thank Dr.-Ing. Barry Linnert, who reviewed what I wrote and helped me make the topic more concrete. He was a very helpful and supportive supervisor throughout this thesis.

I thank Dr. Arman Khalatyan, one of the administrators of the HPC system at the Leibniz Institute for Astrophysics Potsdam, for giving me a lot of information about the system, so that I could make claims that are related to reality.

I also thank Prof. Dr.-Ing. Jochen Schiller for taking on the role of second examiner.

Finally, I thank my family and friends for supporting me throughout my studies, not only during this thesis.

Contents

Eidesstattliche Erklärung i

Acknowledgments ii

1 Introduction 1
- 1.1 Motivation . . . 1
- 1.2 Problem and Research Questions . . . 2
- 1.3 Approach and Contributions . . . 3
- 1.4 Thesis Structure . . . 3

2 Background and Related Work 4
- 2.1 Batch Scheduling in HPC Clusters . . . 4
- 2.2 Checkpointing and Preemption . . . 5
- 2.3 Electricity Prices as a Scheduling Signal . . . 5
- 2.4 Related Work . . . 6
- 2.5 Relation to Kammeyer's Scheduler . . . 7
- 2.6 Research Gap . . . 7

3 System Model 8
- 3.1 Cluster and Job Model . . . 8
- 3.2 Job Profiles . . . 9
- 3.3 Execution States and Checkpointing . . . 9
- 3.4 Electricity-Cost Model . . . 10
- 3.5 Deadlines and Quality of Service . . . 11
- 3.6 Assumptions . . . 12

4 Scheduler Design 13
- 4.1 Common Simulation Loop . . . 13
- 4.2 FCFS Baseline . . . 13
- 4.3 EASY Backfilling Baseline . . . 14
- 4.4 Profile- and Price-Aware Pause/Resume Policy . . . 14
  - 4.4.1 Elements of a Decision . . . 15
  - 4.4.2 Question 1: Urgency . . . 15
  - 4.4.3 Question 2: Price . . . 16
  - 4.4.4 Question 3: Capacity and Priority Order . . . 19
- 4.5 Complexity and Expected Behavior . . . 19

5 Evaluation **20**
5.1 Prototype and Validation . . . 20
5.2 Workloads and Price Data . . . 21
5.3 Experiment Design and Metrics . . . 22
5.4 Does P3R Shift Work to Cheaper Hours? . . . 24
5.5 What Does It Cost Users? . . . 27
5.6 When Does It Help or Fail? . . . 28
5.7 Robustness . . . 30

6 Discussion **34**
6.1 Interpretation . . . 34
6.2 Savings at the Scale of the AIP Cluster . . . 36
6.3 Limitations . . . 37

7 Conclusion and Future Work **40**
7.1 Answers to the Research Questions . . . 40
7.2 Recommendations for the AIP Cluster . . . 41
7.3 Future Work . . . 41

Bibliography **44**

A Use of Tools and AI Assistance **47**
A.1 Documentation of AI Use . . . 47
A.2 Reflection on AI Use . . . 48

B Results of the 60-Day Windows **49**

Chapter 1

Introduction

1.1 Motivation

The Leibniz Institute for Astrophysics Potsdam (AIP) runs its simulations and data reductions on an in-house compute cluster of ∼6,600 CPU cores.[1] Like most high-performance computing (HPC) systems, it is operated by a batch scheduler, in this case Slurm [26]: users submit jobs that request a number of cores and, if needed, other resources such as GPUs for a number of hours. The scheduler decides when each job starts. It bases that decision on the queue order, the free capacity, and the requested runtime. At AIP the queue order follows a fair-share priority, which favors users who have recently used less than their share of the cluster [17]. It does not consider when the electricity for the work is cheap.

That price is not constant. On the German day-ahead market, every hour of the following day has its own price. In 2024 the hourly price for the Germany/Luxembourg zone ranged from −135 to 936 EUR/MWh around a mean of 78.5 EUR/MWh, and it was negative in 457 hours [3]. On the median day, the most expensive hour cost about 98 EUR/MWh more than the cheapest, more than the annual mean price itself. A computation that runs at noon and the same computation run at three in the morning can therefore cost very different amounts, although the scheduler treats them as identical.

Jobs also differ in how urgent they are. Some users need their results right away, for example to debug their code. Others submit a long simulation and need the output only by the next morning or in a few days. The time between the earliest moment a job could finish and the moment its result is needed is the job's slack. A job without slack has to run straight through. A job with slack can sit out an expensive hour and continue in a cheaper one, as long as it still finishes in time. A scheduler that knew the slack of every job could spend it where electricity is cheap and leave the urgent jobs untouched.

Jobs are not uniform over time either. A simulation may start with a compute-heavy initialization, pass through a phase dominated by I/O, and end with another burst

[1] The core count, the processor models, the job size limit, the use of Slurm and of fair share, and the information behind the workload model of Section 5.2 are from personal communication with the administrator of the AIP compute cluster, 2026. The institute's public web page still lists an earlier, smaller configuration.

of computation. A job runs on one or more compute nodes, the individual servers of the cluster, each with its own cores and memory. These nodes stay reserved for the whole job, but they draw most of their power only in the active phases. A scheduler that knew both the price of every hour and the activity of every job could move the active phases of a job into cheaper hours, for example by pausing a checkpointable job during an expensive afternoon and resuming it at night.

Moving work has a price of its own. A paused job waits longer and finishes later, and the users of a cluster like the one at AIP care more about their results than about the electricity bill. Batch systems, however, do not schedule toward deadlines. Slurm, for example, accepts a deadline only as a condition for removing a job that can no longer finish in time [19]. To measure what late results cost, this thesis gives every job a completion deadline derived from how long its user is willing to wait (Section 3.5) and counts every job that misses it. Price awareness is only worth having if it saves money without an unacceptable loss of service, and whether it does depends on how busy the cluster is, how much the price varies, and how much slack the jobs have.

1.2 Problem and Research Questions

Established batch schedulers such as first-come, first-served (FCFS) scheduling and EASY backfilling optimize queue order and utilization. Price-aware HPC schedulers exist, but they choose when a job starts; once started, it runs to completion [25, 15, 11]. Pausing running jobs to follow a time-varying signal has been studied mainly for carbon-aware cloud workloads without capacity limits [24]. What remains open is whether a scheduler that also knows how a job's activity changes from hour to hour, and may pause the job while it runs, can buy that activity more cheaply, and what the users pay for it in waiting time and missed deadlines (Section 2.6).

This thesis asks the following central question:

> Can a scheduler that knows the hourly electricity price and the hourly activity profile of each job reduce the price paid for active computation, compared with FCFS and EASY backfilling, without unacceptable losses in waiting time and deadline compliance?

What counts as unacceptable is an operator's choice that this thesis does not make. It measures the losses so that an operator can weigh them against the saving (Section 5.6). It is broken down into three questions, which structure the results in Chapter 5:

- Q1 Does such a scheduler shift active work to cheaper hours, and by how much does the paid price fall?
- Q2 What does the shift cost users in waiting time, suspension, and deadline misses?
- Q3 Under which cluster loads, price conditions, and parameter settings does it help, and when does it fail?

1.3 Approach and Contributions

To answer these questions, this thesis builds an hourly simulator of a homogeneous 64-node cluster and implements three policies in it: FCFS, EASY backfilling, and a profile- and price-aware pause/resume policy, called P3R in the following, that pauses and resumes checkpointable jobs at hourly boundaries. P3R plans each job's remaining active hours against the known price window and weighs the possible saving against the job's deadline with a single parameter λ. All three policies run on the same synthetic workloads and on the 2024 Germany/Luxembourg price trace, and every comparison is paired on identical jobs.

The thesis makes three contributions. First, it specifies P3R, a pause/resume policy that plans the remaining active hours of each job against the price window with an exact dynamic program over the ordered profile steps, and that trades the saving against the job's deadline with one parameter (Chapter 4). Second, it provides a tested hourly simulator and a paired evaluation design in which all policies see identical jobs and prices, with every reported number traceable to committed simulation runs (Chapter 5). Third, it measures when hourly price shifting pays off. P3R pays 29 % less per megawatt-hour of active computation than EASY at low load and 13 % less at medium load. It buys this with later results and frequent interruptions at low load, and at medium load also with missed deadlines, which a moderate urgency setting avoids while keeping part of the saving. It gains less than 1 % once the offered load exceeds the cluster's capacity (Chapters 5 and 6). Most of the saving comes from shifting work in time, and knowing the job profiles adds 1.3 points at low load.

The study is deliberately idealized. Checkpointing costs neither time nor energy, and the scheduler knows the prices of the whole year in advance. The results are therefore an upper bound on what hourly price shifting can achieve, not a prediction for a production system. One assumption works against price shifting: a paused job keeps its nodes, so other jobs cannot use them, which limits the saving when the cluster is busy. The thesis does not model forecast errors, heterogeneous hardware, fair-share priorities, or the fixed parts of an electricity bill such as grid fees and taxes, and it does not propose a Slurm plugin. Section 3.6 lists every assumption, and Chapter 6 returns to the AIP cluster to estimate what the measured effects could mean at its scale.

1.4 Thesis Structure

Chapter 2 introduces batch scheduling, checkpointing, and electricity prices as a scheduling signal, and positions the thesis within related work. Chapter 3 defines the cluster, the job profiles, the cost model, and the assumptions. Chapter 4 specifies the two baselines and P3R. Chapter 5 describes the experiments and answers Q1 to Q3. Chapter 6 interprets the results, estimates their meaning for the AIP cluster, and discusses the limitations, and Chapter 7 concludes. Appendix A documents the AI tools used for this thesis and the errors that AI-assisted reviews found, and Appendix B lists the results of the two 60-day windows.

Chapter 2

Background and Related Work

The central question of Chapter 1, whether a scheduler that knows the hourly electricity price and the hourly activity of each job can buy computation more cheaply without unacceptable losses in service, touches three topics: batch scheduling on HPC clusters, the checkpointing mechanisms that allow running work to be interrupted, and electricity prices that change by the hour. This chapter introduces each topic as far as the thesis builds on it and then positions the thesis within prior work. The formal definitions follow in Chapter 3.

2.1 Batch Scheduling in HPC Clusters

An HPC cluster is a set of *compute nodes*, each a server with its own processors and memory, joined by a fast network and usually shared by many users. The work that users want done, typically a simulation or an analysis program together with its input data, is submitted as a *batch job*. The user names the program, requests a number of nodes, and states an estimated runtime. A resource manager such as Slurm [26] queues the job until the requested nodes are available [5]. The scheduler pursues several objectives at once. It should keep the expensive machine busy, keep waiting times short, and give users predictable and fair service. The user-facing part of these guarantees is commonly called quality of service (QoS). The objectives conflict [5]. Packing the machine tightly favors small, short jobs, while strict fairness can leave nodes idle.

The simplest queue discipline is *first-come, first-served* (FCFS). Jobs start strictly in submission order. FCFS is transparent and fair in arrival order, but it suffers from head-of-line blocking [5]. While a wide job at the queue head waits for enough free nodes, smaller jobs behind it could run on the idle capacity but are not allowed to.

Backfilling addresses this by letting smaller jobs from further back in the queue run on the idle nodes while the head waits. The EASY scheme, introduced by Lifka for the IBM SP system at Argonne National Laboratory [12], keeps FCFS order for the head of the queue. It computes the earliest time at which the first blocked job can start, holds that time as a reservation, and lets any later job start ahead of the head if doing so does not push the reservation back. Backfilling raises utilization enough that it has become the standard in production schedulers, including Slurm's backfill plugin [20], and its behavior is well studied [23, 22]. It depends on runtime estimates, because reservations

are computed from the runtimes of running and queued jobs. The estimate comes from the user. At submission, a job carries a requested time limit, and the resource manager kills the job when it runs over, so users tend to state generous limits rather than their best guess. Mu'alem and Feitelson studied how sensitive backfilling is to the accuracy of these estimates on IBM SP2 traces and found, counterintuitively, that it worked better when users overestimated the runtime by a substantial factor [14]. This thesis assumes exact runtimes for all policies (Assumption 2). This is a simplification, not a best case for the baselines.

Scheduling research evaluates such policies on synthetic or archived workloads [6]. The usual metrics are the *waiting time* of a job from submission to start, its *response time* from submission to completion, its *slowdown*, and the cluster's *utilization*, the share of all node-hours that were allocated to jobs. The slowdown is the response time divided by the runtime, usually with a lower bound on the runtime so that very short jobs do not dominate the average. Chapter 5 states the exact form in which each of them is measured here.

2.2 Checkpointing and Preemption

P3R, the policy studied in this thesis, pauses running jobs during expensive hours and resumes them during cheaper ones. Pausing a running HPC job in this way requires saving enough state to resume it later. Application checkpointing writes the relevant memory and file state to stable storage, either coordinated by the application itself or, less efficiently, transparently by the operating system or a checkpoint/restart library. Checkpointing costs time, I/O bandwidth, and storage, and not every application supports it. *Scheduler preemption* is the resource manager's ability to stop a running job in favor of other work. It acts on whole jobs rather than on processes. The resource manager suspends, requeues, or cancels the job and reclaims its nodes [18]; the operating system's time slicing of processes within one node is a different mechanism and plays no role here. Preemption is only as good as the checkpoint mechanism underneath, so production systems preempt rarely and selectively.

This thesis uses an idealized form of both. Every job in the focused workload is checkpointable at hourly boundaries, and a checkpoint costs neither time nor energy (Assumptions 4–5). Modeling the checkpoint as costing nothing separates the value of shifting execution in time from the cost of the mechanism that enables it. Chapter 3 states these assumptions and Chapter 6 discusses their consequences.

2.3 Electricity Prices as a Scheduling Signal

Wholesale electricity in Europe is traded, among other venues, in day-ahead auctions that publish an hourly price for every hour of the following day in advance [4]. These wholesale prices vary with demand and with the availability of renewable generation. Electricity in hours with strong wind and solar feed-in can cost several times less than in scarcity hours of the same day, and the price can even turn negative. Most end consumers never see this variation, because they buy at fixed retail tariffs. A large consumer whose procurement contract is indexed to the spot market, however, pays a

different price for the same energy depending on *when* it is consumed. That exposure is what the thesis investigates.

The German Bundesnetzagentur republishes market results on its SMARD platform [3]. This thesis uses the SMARD hourly day-ahead price trace for the Germany/Luxembourg bidding zone for the year 2024 as its price input. Three signal qualities must be distinguished. The historical record is known truth, day-ahead prices are known roughly one day ahead, and longer-horizon forecasts are uncertain. The simulator treats the historical trace as a fully known signal over the entire horizon (Assumption 3). Forecast generation and forecast error are left to an external component and remain outside the scope of this thesis.

2.4 Related Work

Three research lines frame the thesis. For each, this section states what the thesis borrows and what it leaves out.

Classical parallel-job scheduling. The FCFS and EASY-backfilling literature discussed in Section 2.1 provides the comparison baselines. The thesis borrows the established policies and evaluation metrics. Refinements of backfilling beyond EASY, such as conservative variants and lookahead packing [22], are not considered. The role of the baselines is to represent standard production practice.

QoS- and deadline-oriented resource management. Linnert et al. [13] argue that the conventional resource-manager job model, which consists of requested nodes, an estimated runtime, and little else, is too coarse for deadline-oriented HPC resource management, and that richer models of runtime behavior enable dynamic allocation decisions. Earlier work on advance reservations makes a related point. When service guarantees matter, future commitments must be planned, and present capacity alone is not enough [21]. The thesis borrows this perspective directly. The hourly active-node profile of Section 3.2 is a lightweight instance of a richer job-behavior model, and completion deadlines provide the QoS pressure. Process-level application models, migration, and the distributed-resource setting are left out. The profile reduces runtime behavior to one integer per hour.

Energy- and price-aware scheduling. GreenSlot schedules batch jobs in a datacenter with on-site solar generation, delaying jobs within deadlines to align execution with green or cheap energy [9]. Yang et al. choose which waiting HPC jobs to start in on-peak and off-peak hours according to their power draw [25]. Murali and Vadhiyar exploit day-ahead electricity markets when choosing execution slots for HPC jobs [15]. Closest to this thesis, Kammeyer develops backfill-based scheduling for time-dependent optimization goals such as electricity price, renewable share, and CO_2 intensity, using a configurable lookahead window [11]; the next section compares that work with this thesis. All of these act on *start times*. A job, once started, runs to completion. The thesis borrows the price-signal framing and the deadline-bounded delay idea and adds the ability to *pause running jobs*. With checkpointable jobs, a scheduler can react to

price valleys and peaks mid-execution rather than only at job start. The price for this is suspension effects that start-time approaches do not have.

Interrupting running jobs according to a time-varying signal has been studied outside HPC price scheduling. Wiesner et al. split interruptible jobs across the lowest-carbon half-hour slots before a per-job deadline [24]. Their scenarios, nightly batch jobs and machine-learning training in a cloud, assume unlimited capacity and differ from HPC batch scheduling, where jobs compete for a fixed number of nodes. In HPC demand response, Ahmed et al. terminate running jobs and rerun them later to meet a reduced power limit, without checkpoints or deadlines [1]. This thesis combines what these works treat separately: a market price signal, checkpoint-based pause and resume at hourly boundaries, per-job deadlines, and many jobs competing for a fixed number of nodes.

2.5 Relation to Kammeyer's Scheduler

This thesis started with Kammeyer's dissertation on a digital twin for energy-aware HPC at Freie Universität Berlin, and in particular with its concluding chapter [11, Ch. 8]. It is also the closest prior work. Both use SMARD prices and compare against backfilling, and both ask how much a scheduler saves by running work when electricity is cheap.

Kammeyer's scheduler extends backfilling with a lookahead window of several days and gives each job the cheapest start time it finds in that window [11, Ch. 6]. A node power model with CPU frequency selection estimates what each job consumes on a heterogeneous cluster. He evaluates the scheduler against 2023 prices, carbon intensity, and renewable share, and measures the effect on users through response time, bounded slowdown, and utilization.

The main difference is what happens after a job starts. In Kammeyer's scheduler it runs to completion. Under P3R it can be paused and resumed later, which leads to three differences in the design. First, a scheduler that moves single hours has to know which hours of a job actually compute, so this thesis describes a job by its hourly active-node profile instead of its power draw. Second, it has to plan those hours against the price window, which the dynamic program of Chapter 4 does. Third, pausing delays a job that is already running, so the risk is a late finish rather than a late start. Completion deadlines are therefore the service constraint here, traded against the saving through a single tuning parameter that Chapter 4 introduces.

2.6 Research Gap

None of the work discussed above brings together, in one controlled and reproducible setting, (i) a per-job activity profile that separates reserved from active nodes, (ii) hourly electricity prices known over a long lookahead window, (iii) completion deadlines as the service constraint, and (iv) transparent hourly pause/resume decisions. This thesis studies that combination. None of the four parts is new on its own. The contribution is a prototype evaluation that shows when the combination pays off and what it costs in service quality.

Chapter 3

System Model

This chapter defines the formal model in which all later claims are valid: the cluster, the jobs and their hourly activity profiles, the execution states, the electricity-cost model, and the QoS constraints. It ends with the complete list of simplifying assumptions (Section 3.6). Later chapters refer back to that list instead of restating individual assumptions.

3.1 Cluster and Job Model

Time is discrete and measured in hours. The simulation covers a horizon of T hours, $t \in \{0, 1, \ldots, T-1\}$, determined by the length of the electricity-price trace. All scheduling decisions are made at hour boundaries.

The cluster consists of N identical nodes. The experiments in this thesis use $N = 64$. There is no further resource dimension. Memory, network, and storage are not modeled, and every job fits on any subset of nodes of sufficient size.

A job j is described by the tuple

$$j = \left(t_j,\ r_j,\ n_j,\ w_j,\ a_j\right), \tag{3.1}$$

where

- t_j is the submit time (hour of arrival),
- r_j is the runtime in whole hours,
- n_j is the number of requested nodes,
- w_j is the maximum acceptable waiting time in hours, and
- a_j is the activity profile defined in Section 3.2.

Runtimes are assumed to be known exactly (Assumption 2). Jobs belong to one of three runtime classes, short (1–6 h), medium (6–48 h), and long (48–336 h), so the workload spans jobs from interactive debugging runs to simulations lasting up to 14 days. The maximum wait w_j is a class-dependent multiple of the runtime, $w_j = \lceil f \cdot r_j \rceil$, with smaller relative factors f for longer jobs. A two-week simulation may tolerate days of

absolute delay, but not multiples of its own runtime. The exact generator distributions are experimental parameters and are given in Chapter 5.

Capacity feasibility is defined on requested nodes. At every hour, the requested nodes of all jobs holding a reservation must not exceed N. Whether a job fits therefore depends only on n_j, never on how many of those nodes it actively uses.

3.2 Job Profiles

A flat job model that reserves n_j nodes for r_j hours implicitly assumes that every reserved node is equally busy during every hour of the job. Real applications behave differently. Initialization, I/O, and post-processing phases often use only part of the allocation, while compute phases use all of it. The job profile makes this visible to the scheduler at low modeling cost. It follows the argument of Linnert et al. [13] that resource management benefits from richer models of a job's runtime behavior. The hourly active-node count is the most lightweight such model that still affects electricity cost.

The profile of job j is a sequence of active-node counts, one per runtime hour:

$$a_j = \big(a_j(0), a_j(1), \ldots, a_j(r_j - 1)\big), \qquad 0 \le a_j(k) \le n_j. \tag{3.2}$$

Here $a_j(k)$ is the number of reserved nodes that are actively computing during the k-th hour of the job's execution. A profile is valid only if it has exactly r_j steps and satisfies the bound in Equation (3.2). The simulator rejects invalid profiles at construction time.

Two points delimit what a profile is *not*. First, $a_j(k)$ counts whole active nodes, not CPU-utilization percentages or instruction rates. It is a coarse activity measure by design. Second, the profile does not release capacity. Reserved but inactive nodes remain allocated to the job (Assumption 6). The profile affects only energy and cost accounting and the price-aware scheduling decisions built on it.

The synthetic workload draws profiles from seven shape families: flat, front-loaded, back-loaded, valley (active, quiet, active), peak (quiet, active, quiet), ramp-up, and ramp-down. Figure 3.1 shows one example of each family for a job with $n_j = 8$ nodes and a runtime of 48 h. The front-loaded, back-loaded, valley, and peak shapes switch between the full allocation n_j and $\lfloor n_j/2 \rfloor$; the two ramps move linearly between these values. Jobs that run for a single hour are always flat. The shapes matter because two jobs with identical r_j and n_j can have different cost-optimal placements in time. A back-loaded job wants its *final* hours in cheap periods, a front-loaded job its *first* hours. A flat model cannot express this difference.

3.3 Execution States and Checkpointing

Under the baseline policies, a job is either *pending*, *running*, or *completed*. Once started, it runs continuously for r_j hours.

Under P3R, checkpointable jobs additionally support an hourly pause/resume cycle. Each job tracks a progress counter $g_j \in \{0, \ldots, r_j\}$, the number of profile steps already executed. In each hour, a started job either *runs*, which executes step $a_j(g_j)$ and advances g_j by one, or is *paused*, which leaves g_j unchanged. The job completes

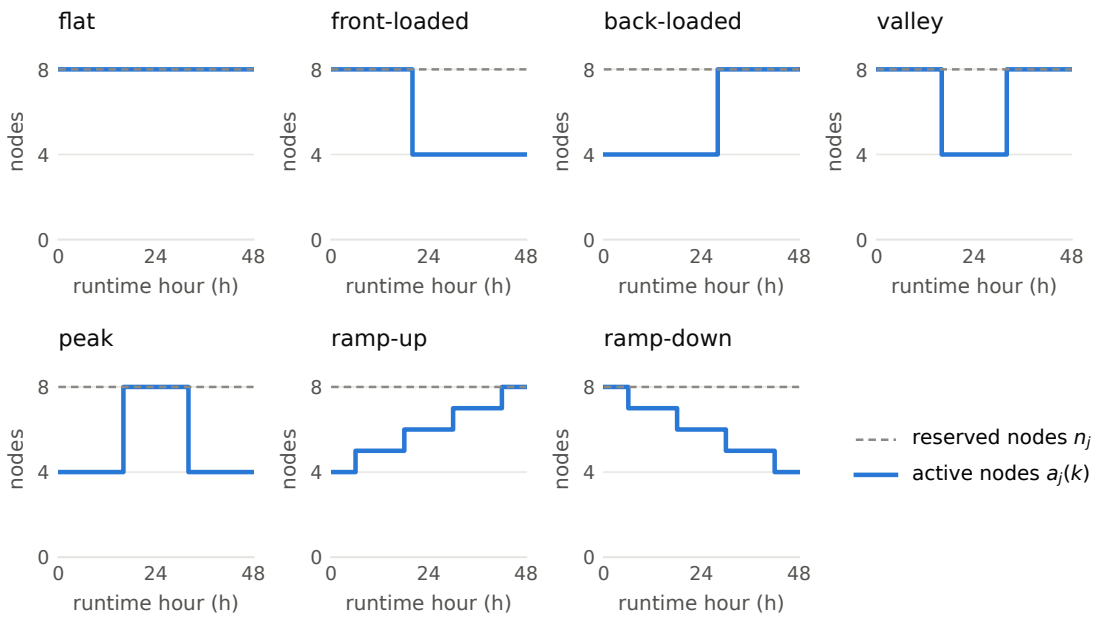


Figure 3.1: *The seven synthetic profile shape families for an example job with* $n_j = 8$ *and* $r_j = 48\,h$. *Dashed: reserved node count* n_j*; solid: active-node count* $a_j(k)$.

when $g_j = r_j$. Pausing is idealized (Assumptions 4 and 5). It is possible at every hour boundary and costs neither time nor energy. A paused job retains its full reservation of n_j nodes (Assumption 6), so pausing reduces active energy consumption but frees no capacity.

All jobs in the evaluated workload are checkpointable, which keeps the comparison focused on a single experimental dimension. The scheduler itself also handles non-checkpointable jobs (Chapter 4). Mixed workloads are a subject for future work.

3.4 Electricity-Cost Model

The electricity price is an hourly sequence $p(t)$ in EUR/MWh, taken from the historical 2024 German/Luxembourg day-ahead market trace (Section 2.3). The full trace is known to the scheduler in advance (Assumption 3).

Only *active* nodes incur cost. For a job that starts at hour s and runs without interruption, the active execution cost is

$$C_j(s) \;=\; \sum_{k=0}^{r_j-1} a_j(k) \cdot P_{\text{node}} \cdot \frac{p(s+k)}{1000}, \tag{3.3}$$

where

- $a_j(k)$ is the number of active nodes in profile hour k,
- P_{node} is the constant power drawn per active node in kilowatts; the experiments use $P_{\text{node}} = 1\,\text{kW}$ (Assumption 7),
- $p(s+k)$ is the electricity price in EUR/MWh at hour $s+k$, and

- the division by 1000 converts kilowatt-hours priced in EUR/MWh into euros.

Negative prices enter the sum with their sign. An active node-hour placed in such an hour lowers the job's cost, as it would for a consumer buying at the wholesale price. A real consumer would still pay grid fees, levies, and taxes in these hours, which the model excludes (Assumption 8).

For a job executed with interruptions, the sum runs over the hours in which the job actually executed, weighting each executed profile step with the price of the hour it was placed in. Because cost is weighted by $a_j(k)$ rather than n_j, two jobs with equal reservations can have different costs, and the same job can become cheaper or more expensive depending on which hours its high-activity steps occupy.

The model covers only *job-attributable active execution cost*. Real nodes also draw idle and base power, and a complete facility bill would include cooling and infrastructure. The cluster is assumed to remain powered on throughout the simulation period. Idle and base power then form a constant background cost that is identical for all compared policies and cancels in the comparison (Assumption 8). All cost figures in this thesis are active-execution costs, not a data-center electricity bill.

3.5 Deadlines and Quality of Service

The maximum wait w_j induces the QoS constraint, and it has the same meaning under every policy. A job must start no later than its *latest start time*

$$\ell_j = t_j + w_j, \tag{3.4}$$

or, equivalently, complete no later than its *completion deadline*

$$d_j = t_j + w_j + r_j. \tag{3.5}$$

For the baselines, which run a job continuously once started, the two statements coincide. For the preemptive policy, which can pause a job after it has started, only the completion form is meaningful: a job could start punctually and then be suspended indefinitely, so the constraint is placed on completion.

A job is never abandoned for missing its constraint. Under every policy it stays in the system until it has run, and a job that completes after d_j, or that cannot complete before the end of the simulation, counts as one QoS violation. Lateness is recorded, not punished by cancellation. Production resource managers also keep a waiting job in the queue by default. Slurm removes it only if the user has set a deadline (Section 1.1).

The end of the simulation follows the same rule for every policy. A job that cannot finish before the horizon still starts when it fits, runs until the horizon, is charged for the hours it ran, and counts as a violation. It does not hold up the jobs behind it. Waiting time, response time, and slowdown are computed over the jobs that complete. All policies receive the same jobs and none discards any, so violation counts differ between policies only because the policies schedule differently.

3.6 Assumptions

The following list is the authoritative statement of the model's idealizations. Each assumption widens the gap between the simulated system and a production system. Chapter 6 discusses the consequences.

1. **Hourly granularity.** Decisions, arrivals, progress, and prices are aligned to hour boundaries. Sub-hourly effects do not exist in the model.
2. **Perfect runtime estimates.** The scheduler knows each job's exact runtime r_j at submission. Real user estimates are notoriously inaccurate. This is a simplification for all three policies. P3R relies on it most, since its plans reach up to the deadline. For EASY it is not a best case, since backfilling can work better with overestimated runtimes [14].
3. **Perfect price knowledge.** The full hourly price trace is known for the entire horizon. Day-ahead markets publish prices roughly one day ahead. Longer-horizon knowledge would in practice require a forecasting component, which is out of scope.
4. **Universal hourly checkpointability.** Every job in the focused workload can be paused at any hour boundary and resumed later.
5. **Zero checkpoint overhead.** Pausing and resuming consume no time and no energy, and no progress is lost.
6. **No capacity release.** Paused jobs retain their full node reservation. Pausing saves energy cost but never frees nodes for other jobs.
7. **Constant active-node power.** Every active node draws $P_{\text{node}} = 1\,\text{kW}$ regardless of workload. The active-node count is a proxy for power, not a measured power model.
8. **Active-job cost only.** Idle and base power, cooling, and infrastructure costs are excluded. Reported costs are job-attributable active execution costs.
9. **Homogeneous single-resource cluster.** All nodes are identical, and nodes are the only modeled resource. Fair-share priorities, heterogeneity, thermal, network, and storage effects are absent.

Assumptions 3–5 all favor price shifting, and Assumption 2 helps it as well: the scheduler sees every future price, and pausing is always possible and costs neither time nor energy. Under them, the model estimates an *upper bound* on the value of shifting active execution in time under perfect hourly checkpointing. Assumption 6 works against price shifting, since a paused job's nodes stay unusable for other jobs. This framing is used throughout the thesis.

Chapter 4

Scheduler Design

This chapter specifies the three compared policies exactly as implemented in the simulator: the FCFS and EASY-backfilling baselines and the proposed profile- and price-aware pause/resume policy. The baselines are standard. Their precise rules still need stating, because the evaluation results depend on those details.

4.1 Common Simulation Loop

All policies run inside the same hourly loop. For each hour $t = 0, \ldots, T-1$:

1. jobs with submit time $t_j = t$ enter the system;
2. the policy decides which jobs start, run, pause, or resume in this hour, subject to the capacity constraint on requested nodes;
3. selected jobs execute one profile step, and the simulator records cost, energy, waiting, and state statistics.

The scheduler is online with respect to arrivals. At hour t it knows only the jobs with $t_j \leq t$; a job submitted at $t+1$ is invisible until then. Its foresight is limited to the price trace and to the runtimes and profiles of the jobs it already knows (Assumptions 2 and 3). Within one policy, the loop is deterministic. Given the same workload and price trace, it reproduces the same schedule. All policies of one experiment receive identical workloads and prices, so observed differences come from the policies alone.

4.2 FCFS Baseline

First-come, first-served keeps the pending queue in submission order and never reorders it. Each hour, it starts the job at the head of the queue if that job's requested nodes are free, and repeats this until the head no longer fits. Jobs behind a blocked head never start before it. Started jobs run continuously to completion. A pending job whose latest start time ℓ_j passes stays in the queue, runs when it fits, and counts as a QoS violation (Section 3.5). The same holds for a job that cannot finish before the end of the simulation. It still starts when it reaches the head and fits, runs until the end, and counts as a violation.

FCFS is the sanity baseline. It is deterministic and fair in arrival order, but it wastes capacity whenever a wide head job blocks narrower jobs that could run immediately.

4.3 EASY Backfilling Baseline

EASY backfilling extends FCFS with one conservative optimization. Jobs may be pulled forward past a blocked queue head, provided they do not delay the head [12]. Each hour, the policy proceeds in three steps.

1. **Start in order.** Like FCFS, start queue-head jobs while they fit.
2. **Reserve for the blocked head.** For the first job that does not fit, compute the earliest hour at which enough nodes will be free for it, given the jobs currently running, and hold this hour as its reservation.[1]
3. **Backfill.** Consider the remaining pending jobs in queue order. A candidate that fits now starts if, and only if, starting it does not push the head's earliest feasible start beyond the held reservation. The policy checks this directly. It recomputes the head's reservation with the candidate hypothetically running and admits the candidate only if the recomputed reservation is not later than the held one.

Backfilling needs runtime estimates to compute reservations. Under the perfect-estimate assumption (Assumption 2), EASY computes its reservations from exact runtimes. This is a simplification rather than a best case, since backfilling can do better with overestimated runtimes [14]. Like FCFS, EASY is non-preemptive and price-blind. It improves packing, but it pays whatever the electricity price happens to be while jobs run.

4.4 Profile- and Price-Aware Pause/Resume Policy

The policy proposed in this thesis is called P3R, short for profile- and price-aware pause/resume. Every hour, for every job that is not yet complete, P3R answers three questions in a fixed order.

Question 1 (urgency). Must this job run now to meet its deadline?

Question 2 (price). If not, is running now cheaper than waiting for a later hour, once the whole remaining profile is taken into account?

Question 3 (capacity). If the job wants to run, does it fit into the free nodes, given the jobs ahead of it?

[1] Near the end of the simulation no such hour may exist. The reservation is then the end of the simulation, so no candidate can delay the head.

4.4.1 Elements of a Decision

A decision about job j at hour t involves the following quantities.

1. *Progress* g_j: the number of profile steps the job has already executed (Section 3.3). For a job that has not started, $g_j = 0$.
2. *Remaining steps* ρ_j: the number of steps still to execute, $\rho_j = r_j - g_j$.
3. *Remaining profile* A: the active-node counts of the steps still to execute, in the order they must execute, $A = (a_j(g_j), \ldots, a_j(r_j - 1))$. A has ρ_j entries, and $A_1 = a_j(g_j)$ is the step that would run now.
4. *Decision window* $W_j(t)$: the hours from now up to the completion deadline d_j (Equation (3.5)), cut off at the end of the simulation:

$$W_j(t) = \{\, t, t+1, \ldots, \min(d_j, T) - 1 \,\}.$$

   Only hours in this window are candidates for the remaining steps.
5. *Window prices* P: the electricity price of every hour in the window:

$$P = \big(p(t), \ldots, p(\min(d_j, T) - 1)\big),$$

   so $P_1 = p(t)$ is the price now.
6. *Urgency parameter* $\lambda \geq 0$: the single tuning knob of P3R. It is fixed for a whole simulation run.

Running example. Job J has runtime $r_J = 6$ and has executed three steps, so $g_J = 3$ and $\rho_J = 3$. Its remaining profile is $A = (4, 2, 4)$: a four-node step, then a two-node step, then a four-node step. Its deadline is five hours away, so the window has five hours, and the prices in these hours are

$$P = (100,\ 40,\ 120,\ 30,\ 90) \quad \text{EUR/MWh}. \tag{4.1}$$

The price now is 100. The cheapest hour in the window is the fourth (30), the most expensive the third (120). The example is used through the rest of this section.

4.4.2 Question 1: Urgency

A job has slack as long as its remaining steps do not fill the window. The urgency test is

$$\text{job } j \text{ is } \textit{mandatory} \text{ at hour } t \quad \Longleftrightarrow \quad \rho_j \geq |W_j(t)|. \tag{4.2}$$

In words: if the number of steps still to execute is at least the number of hours left before the deadline, skipping even one hour makes on-time completion impossible, so the job must run now. A mandatory job skips Question 2 entirely.

Every incomplete job with slack, that is $\rho_j < |W_j(t)|$, goes on to Question 2, which sorts it into one of two classes. It is *opportunistic* if Question 2 says that running now is worthwhile, and *held* if Question 2 prefers to wait. A held job that had already started

is paused for this hour, and the simulator records the suspension. Pausing does not free capacity. The paused job keeps its n_j-node reservation (Assumption 6), so no waiting job can enter the cluster in its place; the only effect of the pause is that the job's current step is not executed and not paid for.

In the running example, $\rho_J = 3$ and $|W_J(t)| = 5$, so J is not *mandatory*. It could skip up to two hours and still finish on time.

Non-checkpointable jobs are absent from the focused workload (Section 3.3) but supported for completeness. Before starting, such a job is held until the delayed-start score $C_j(s) + \lambda\,(s - t_j)\,/\,\max(1, w_j)$ over the start hours s up to its latest start time is smallest for starting now, or until its latest start time forces a start.[2] Once started, it counts as *mandatory* in every later hour, since it cannot be paused.

4.4.3 Question 2: Price

Question 2 compares two plans for the job: run the current step now, or pause for one hour. To compare them, P3R needs to know how cheaply the remaining steps can be placed into the remaining hours. That quantity is defined first.

The best placement cost. For a remaining profile A' with m steps and a price sequence P' with $n \geq m$ hours, define

$$\text{BestCost}(A', P') \;=\; \min_{h_1 < h_2 < \cdots < h_m} \sum_{i=1}^{m} A'_i \cdot P'(h_i). \tag{4.3}$$

In words: choose m distinct hours out of the n available, in increasing order, assign step 1 to the first chosen hour, step 2 to the second, and so on, and add up active nodes times price for every step. BestCost is the smallest such sum over all valid choices. The order constraint $h_1 < h_2 < \cdots$ is essential. Profile steps execute in sequence, so a cheap hour early in the window can only serve an early step. Both BestCost and the scores built on it omit the constant factor $P_{\text{node}}/1000$ from Equation (3.3), since it cancels in every comparison.

In the running example, $A = (4, 2, 4)$ and $P = (100, 40, 120, 30, 90)$. The three steps can be placed into the five hours in $\binom{5}{3} = 10$ ways. The cheapest is hours $2, 3, 4$: the four-node step at 40, the two-node step at 120, the final four-node step at 30, for a total of $4 \cdot 40 + 2 \cdot 120 + 4 \cdot 30 = 520$. Note that the plan puts the small step into the most expensive hour. That is the ordering constraint at work: to reach the cheap fourth hour with the final step, something has to occupy the third hour, and the two-node step is the cheapest thing to put there.

Cost of exhaustive enumeration. Enumerating all placements, as done by hand above, takes $\binom{n}{m}$ sums. For the example this is ten. For a long job with $m = 100$ remaining steps and a window of $n = 300$ hours it is $\binom{300}{100} \approx 10^{81}$, and P3R would need

[2] Here $C_j(s)$ is in euros, including the factor $P_{\text{node}}/1000$ of Equation (3.3), so λ is on a different scale than in the checkpointable rule of Section 4.4.3. The scan considers only start times at which the job finishes within the horizon and does not check capacity; capacity is checked when the job is admitted (Section 4.4.4).

this number for every job in every hour. Dynamic programming reduces it to a table with $(m+1)(n+1)$ entries.

Computing BestCost **by dynamic programming.** Define the partial quantity

$$B(i,h) = \text{the cheapest cost of placing steps } i,\dots,m \text{ into hours } h,\dots,n, \tag{4.4}$$

so that $\text{BestCost}(A',P') = B(1,1)$. B can be filled in backwards, from the last step and the last hour toward the first.

1. **Initialization.** With no steps left there is nothing to pay:

$$B(m+1,h) = 0 \qquad \text{for all } h. \tag{4.5}$$

2. **Induction.** At hour h with step i still to place, there are two choices. Run step i now and continue with the next step in the next hour, or leave hour h idle and try step i in the next hour:

$$B(i,h) = \min\Big(\underbrace{A'_i \cdot P'(h) + B(i+1,h+1)}_{\text{run step } i \text{ at hour } h},\ \underbrace{B(i,h+1)}_{\text{leave hour } h \text{ idle}}\Big). \tag{4.6}$$

   The idle choice is only allowed while enough hours remain for the outstanding steps, that is while $n-h \geq m-i+1$. Otherwise only the run choice exists.

3. **Termination.** $\text{BestCost}(A',P') = B(1,1)$.

Each entry of B is computed once from two already known entries, so the whole table costs at most $(m+1)(n+1)$ operations. Only entries with enough hours left for the outstanding steps are ever needed, which leaves a band of $m(n-m+1)$ of them. For the long job above that is $100 \cdot 201 = 20{,}100$ instead of 10^{81}. The shortcut is valid because the best way to place steps $i,\dots,m$ into hours $h,\dots,n$ does not depend on where steps $1,\dots,i-1$ went, as long as they went before hour h. Each (i,h) subproblem therefore has to be solved only once.

Figure 4.1 shows the table for the running example. Its rows are the remaining steps and its columns the hours. A diagonal arrow stands for running the next step in that hour, a horizontal arrow for staying idle. Following the arrows from $B(1,1) = 520$ gives the optimal plan: idle in hour 1, then run the three steps in hours 2, 3, and 4.

The two plans. With BestCost available, the two plans can be priced. Write $A_{2:}$ and $P_{2:}$ for the remaining profile and the window prices without their first entry.

$$\mathit{run_now} = A_1 \cdot P_1 \ + \ \text{BestCost}\big(A_{2:},\ P_{2:}\big), \tag{4.7}$$

$$\mathit{skip_now} = \text{BestCost}\big(A,\ P_{2:}\big). \tag{4.8}$$

run_now pays for the current step at the current price and places the rest of the profile as well as possible in the remaining hours. *skip_now* pays nothing now, gives up the current hour, and places the whole remaining profile as well as possible in the hours that are left. The difference between the two is exactly the value of the

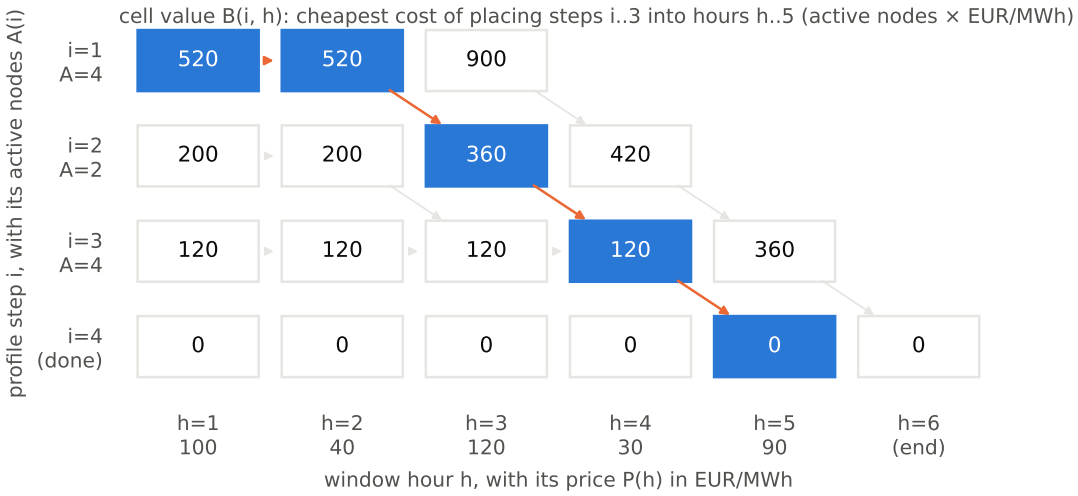


Figure 4.1: *The table $B(i,h)$ of Equation (4.6) for the running example. Diagonal arrow: run; horizontal arrow: idle. The highlighted path is the optimal plan.*

current hour to this job. Both plans come from one table over the whole window: $\mathit{run_now} = A_1 \cdot P_1 + B(2,2)$ and $\mathit{skip_now} = B(1,2)$.

In the running example, running now costs $4 \cdot 100 = 400$ for the current step, and the best placement of $(2,4)$ into $(40,120,30,90)$ is $2 \cdot 40 + 4 \cdot 30 = 200$, so $\mathit{run_now} = 600$. Skipping gives $\mathit{skip_now} = \mathrm{BestCost}\big((4,2,4),(40,120,30,90)\big) = 520$, the placement found above. Both values can be read off Figure 4.1 as $B(2,2) = 200$ and $B(1,2) = 520$. Waiting one hour saves 80.

The urgency penalty. A pure cost comparison would pause a job for an entire day to save a marginal amount, at the expense of waiting time and suspension. The urgency penalty makes this trade-off tunable:

$$\mathit{penalty} \;=\; \lambda \cdot \big(\max P - \min P\big) \cdot \max\big(1,\ \overline{A}\big), \tag{4.9}$$

where $\overline{A}$ is the mean of the remaining active-node counts. The factor $\max P - \min P$ is the price span of the window. It makes the penalty independent of the absolute price level: doubling every price doubles both the saving from waiting and the penalty. The factor $\max(1, \overline{A})$ scales the penalty to the size of the job, so that a wide job and a narrow job are held to the same relative standard. Read together, λ says how large a saving, measured in units of "the largest possible price spread times the job's typical width", is required to justify one hour of additional delay.

The job runs now if

$$\mathit{run_now} \;\le\; \mathit{skip_now} + \mathit{penalty}. \tag{4.10}$$

In words: run unless waiting saves more than the penalty.

In the running example, the price span is $120 - 30 = 90$ and the mean active-node count is $(4+2+4)/3 \approx 3.33$, so the penalty is 300λ. Waiting saves 80. With $\lambda = 0$ the job is *held*, since $600 > 520$. With $\lambda = 0.1$ it is still *held*, since $600 > 550$. With

$\lambda = 0.5$ it runs, since $600 \leq 670$. The switch happens at $\lambda = 80/300 \approx 0.27$. This is the behavior λ is meant to produce: with $\lambda = 0$ P3R pauses whenever waiting is even marginally cheaper, and as λ grows, pausing becomes rare and the policy approaches run-when-possible behavior.

One boundary case bypasses the comparison. If the window is empty or the remaining steps fill it completely, the job is *mandatory* and runs. This also covers the case in which *skip_now* would be undefined, because too few hours remain after a pause.

4.4.4 Question 3: Capacity and Priority Order

Questions 1 and 2 decide which jobs *want* to run this hour. Question 3 decides which of them *can*. Started incomplete jobs hold their n_j-node reservations whether they run or pause (Assumption 6), so only the remaining free capacity is available for jobs that have not yet started. Mandatory jobs are served before opportunistic ones. Within each class, jobs are ordered by

1. earlier completion deadline d_j,
2. smaller slack $d_j - t - \rho_j$,
3. lower current-step cost $A_1 \cdot P_1$, and
4. lower job identifier (final deterministic tie-break).

Jobs are admitted in this order. A job already holding a reservation always proceeds. A new job starts only if its n_j nodes fit into the remaining free capacity. Unlike EASY, P3R holds no reservation for a job that does not fit, so a narrower job behind it may start first. A mandatory job that does not fit must wait despite its urgency, because capacity is a hard constraint. Under high load this turns deadline pressure into QoS violations. Eligible jobs that fit always run. P3R never leaves usable capacity idle to wait for cheaper prices unless Question 2 held the job.

4.5 Complexity and Expected Behavior

Per hour, FCFS and EASY spend most of their time on feasibility checks against the running set. EASY also recomputes the head reservation once per backfill candidate. P3R evaluates Equation (4.10) once per eligible job per hour, and each evaluation fills one table of $\rho_j(|W_j| - \rho_j + 1)$ entries, which is small when the job has little slack. P3R uses the exact dynamic program for every job, however long. Simulating a full year of one synthetic workload takes about twenty seconds on a laptop. The evaluation in Chapter 5 needs 360 such runs per experiment, ten random workloads for every combination of load level and policy setting, so this speed is what keeps the parameter sweeps tractable.

The design implies the following behavior. With low λ, P3R chases price valleys aggressively. Suspensions and waiting rise, and paused jobs retain capacity that other jobs cannot use (Assumption 6). With high λ, it approaches a work-conserving scheduler and forgoes most savings. The interesting region, and the subject of the evaluation, lies between these extremes.

Chapter 5

Evaluation

This chapter describes the simulator prototype and how it was validated, the workloads and the price data, and the experiment design and the metrics. It then presents the results organized by research question and closes with five checks of their robustness.

5.1 Prototype and Validation

The simulator behind all results that follow is a single Python module of about 700 lines with no dependencies beyond the standard library. It defines five data types: a `Job` carrying the tuple of Section 3.1, a `JobProfile` holding the active-node sequence, a `ScheduledJob` recording a baseline job's start, a `PreemptiveJobState` tracking progress, first start, completion time, and suspension counters for P3R, and a `PolicyResult` collecting the metrics of one run. One function, `simulate`, executes the hourly loop of Section 4.1 for a given policy, workload, and price trace. A separate report script runs the full experiment grid in parallel and writes the tables and figures used in this chapter.

Validation has three layers. First, the data types reject invalid input at construction time. A profile must have exactly r_j steps and satisfy $0 \leq a_j(k) \leq n_j$, and every job must have a positive runtime and node count. Second, a suite of 32 unit tests checks the behavior that the thesis relies on. Cost accounting must use active rather than reserved nodes, and EASY must never schedule fewer jobs than FCFS on a fragmented queue. P3R must suspend an expensive hour and preserve progress, run an urgent job even when the price is high, never suspend a non-checkpointable job, keep the reservation of a paused job, and run a low-activity step to save a cheap hour for a high-activity step. All three policies must complete the same set of jobs, both on a workload in which a job misses its latest start time and on one in which a job cannot finish before the horizon. The workload generator is tested for ordered load levels, valid profiles, and the 14-day runtime cap.

Third, the run-or-skip decision of Section 4.4.3 is cross-checked against exhaustive enumeration. On 300 random small instances with prices between -150 and 200 EUR/MWh, the decision taken by the simulator's dynamic program equals the decision obtained by enumerating every placement of the remaining steps. For long jobs, where enumeration takes too long, a further test compares the dynamic program with a plain full-table implementation. The simulator's decision function also repro-

Table 5.1: *Load levels: base arrival rate, offered load, and class mix.*

Level	Base rate (jobs/h)	Offered load	Short/medium/long (%)
low	0.16	0.34	50/37/13
medium	0.355	0.77	50/37/13
high	0.58	1.23	50/37/13
mixed	0.46	1.23	45/38/17

duces the running example of Chapter 4, pausing the job at $\lambda = 0.26$ and running it at $\lambda = 0.27$, on either side of the switching point 80/300.

Beyond these tests, the code and the text were checked against each other in an audit of about sixty statements of Chapters 3 and 4, and every number reported in Chapters 5 and 6 was recomputed from the results of the individual simulation runs. Both reviews were AI-assisted, and both found errors that are corrected in this thesis. Appendix A lists them.

The simulator, the experiment scripts, the price data, and the results of every run reported in this thesis are publicly available at `https://github.com/WawasCode/p3r-hpc-scheduling`. The repository holds one folder per run, including the robustness checks of Section 5.7, together with the commands that reproduce them.

5.2 Workloads and Price Data

The simulator runs on two inputs, a synthetic workload and a historical price trace. In the workload, jobs arrive according to a Poisson process whose base rate sets one of four load levels: low, medium, high, and mixed (Table 5.1). The rate is multiplied by 1.25 between 08:00 and 19:00 and by 0.70 otherwise, and by 1.15 on weekdays and 0.75 on weekends, which gives the arrivals a daily and a weekly rhythm like a real submission log. The *offered load* of a level is its requested node-hours divided by the capacity $N \cdot T$. Low and medium stay below capacity. High and mixed exceed it by design, to test the policies under sustained overload, where no scheduler can serve every job. The mixed level also shifts the class mix toward long jobs.

Each job then draws a runtime class, short, medium, or long, with the probabilities of Table 5.1. Table 5.2 lists the distributions within each class. Short runtimes are drawn uniformly from 1 to 6 hours. Medium and long runtimes are drawn from a lognormal distribution and clipped to the class range, and long jobs also have an 8 % chance of running exactly 336 h, so that the 14-day maximum occurs in every workload. Node counts and wait factors are drawn from the values in the table with the listed weights. The *maximum wait* of a job is $w_j = \lceil f \cdot r_j \rceil$ with the class-dependent factor f. Short jobs therefore tolerate a delay of one to three times their runtime. Long jobs tolerate between a quarter of their runtime and their full runtime, which is less relative to their runtime but more in absolute hours. The profile shape is drawn from the seven families of Section 3.2 with weights 25/22/22/11/8/6/6 for flat, front-loaded, back-loaded, valley, peak, ramp-up, and ramp-down.

This thesis prefers synthetic workloads to archived traces for two reasons. They make the load level and the runtime mix explicit experimental variables rather than

Table 5.2: *Synthetic job classes: runtime range, node counts, and wait factors. Weights are in percent.*

Class	Runtime r_j	Nodes n_j (weights)	Wait factor f (weights)
short	1–6 h, uniform	1, 2, 4 (55/35/10)	1, 2, 3 (35/50/15)
medium	6–48 h	1, 2, 4, 8 (35/30/20/15)	0.75, 1, 1.5, 2 (20/35/35/10)
long	48–336 h	4, 8, 10 (40/35/25)	0.25, 0.5, 0.75, 1 (20/45/25/10)

properties of one particular site, and they carry a profile for every job, which no public trace provides. What the choice gives up is realism. The arrival pattern and the class distributions are not measured, because no accounting log of the target cluster was available. They are this thesis's interpretation of conversations with the administrator of the AIP cluster about the jobs it runs. The widest jobs request 10 of the 64 nodes, about a sixth of the cluster, because the AIP cluster limits a job to about 1000 of its cores. They make short jobs the most common class, while long jobs, which run longer and on more nodes, carry most of the work. In the generated workloads long jobs are 13 % of all jobs but 84 % of the requested node-hours, so most of the energy, and most of the absolute slack that price shifting can use, sits in a minority of long jobs. The mixed load raises their share to 17 % of the jobs and 87 % of the node-hours to test whether more long jobs change the result. The results therefore describe the behavior of the policies on this family of workloads, not on any specific cluster. Replaying a real accounting trace is listed as future work (Section 7.3).

Unlike the workload, the price input is historical. It is the SMARD hourly day-ahead price for the Germany/Luxembourg bidding zone for the calendar year 2024. The series covers 8784 hours in UTC without gaps or duplicates and is used as is, without smoothing or scaling. Prices range from −135 to 936 EUR/MWh with a mean of 78.5 EUR/MWh, and they are negative in 457 hours. Figure 5.1 shows the trace and enlarges one week in January and one in July.

The seasonal structure of the trace matters for the evaluation. The mean spread between the most and the least expensive hour of a day is about 55 EUR/MWh in January and February and about 150 EUR/MWh from July to September, when midday solar feed-in produces deep price valleys. In the enlarged January week the price follows a daily wave of about 60 EUR/MWh, while in the July week midday prices fall to zero and evening peaks exceed 200 EUR/MWh. Of the 457 negative hours, 381 fall between April and September. A policy that shifts work into cheap hours therefore has roughly three times as much to gain per day in summer as in winter, which is why the main experiment covers the whole year rather than one window.

5.3 Experiment Design and Metrics

Each experiment is a grid over load level, policy, and seed. For every combination of seed and load, one workload is generated and given unchanged to all policies, so that the policies are compared on identical job sets and identical prices. The three policies are FCFS, EASY, and P3R at seven values of the urgency parameter, $\lambda \in \{0, 0.01, 0.05, 0.1, 0.25, 0.5, 1\}$, which spans the range from purely price-seeking

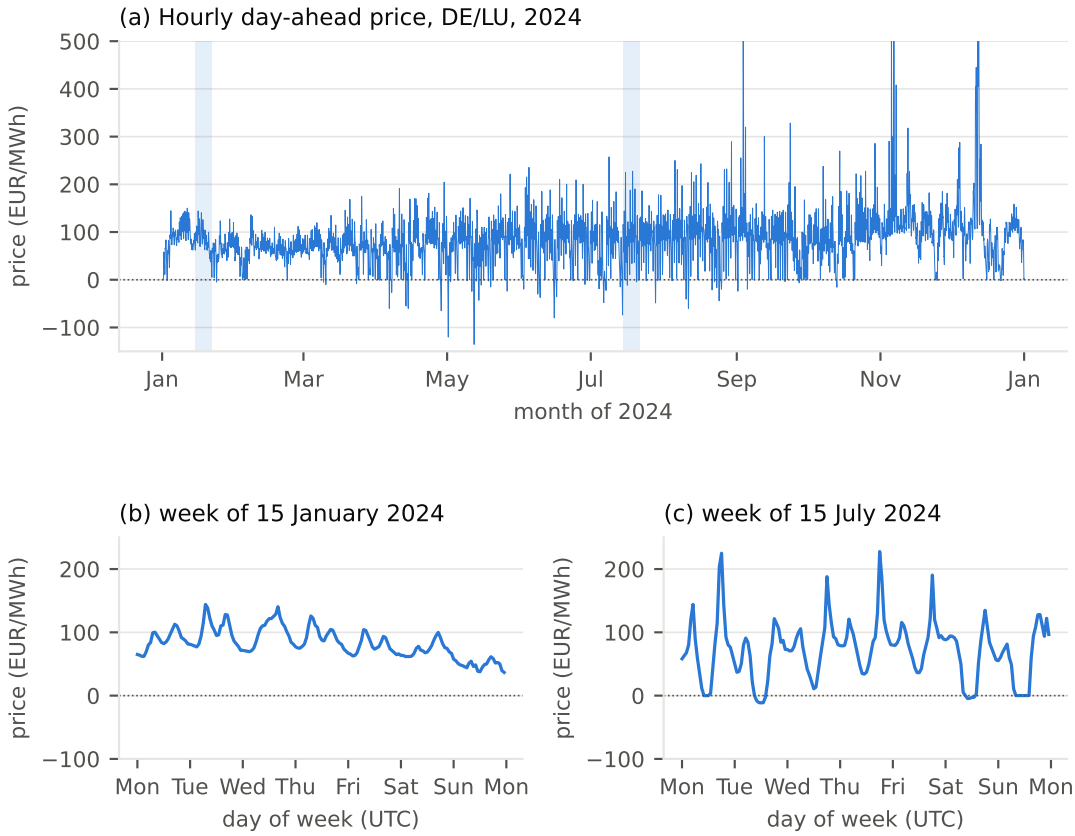


Figure 5.1: *The 2024 SMARD day-ahead price trace for the DE/LU bidding zone (top, clipped at 500 EUR/MWh) and two enlarged weeks (bottom, shaded in the top panel).*

to run-when-possible behavior. Each configuration runs with ten seeds, which gives $4 \times (2 + 7) \times 10 = 360$ simulation runs per experiment. All results are reported as means over the ten seeds together with 95 % confidence intervals, written as mean $\pm$ half-width. A difference is called distinguishable from zero when its interval does not include zero. Because ten values are few, the intervals use the t distribution, the standard choice for small samples, with nine degrees of freedom, one less than the number of seeds. The half-width is therefore 2.262 times the standard error of the ten per-seed values. The main experiment covers the full 8784-hour year, and two 60-day windows, January to February and July to August, isolate the seasonal effect of the price signal (Section 5.7).

Since jobs arrive during the whole horizon and the simulation stops at its end, jobs submitted near the end may be unable to finish. Every policy treats such a job the same way. It starts when it fits, runs until the horizon, pays for the hours it ran, and counts as a violation (Section 3.5). The effect is small over a year and larger in the 60-day windows, and it shows up in the completion and violation counts.

Completions and violations are two of the metrics below, all of which are defined as the simulator implements them. For one run, let J be the number of submitted jobs and C the number of completed jobs.

- **Total cost** (EUR): the active execution cost of Equation (3.3), summed over all

executed step-hours.

- **Normalized paid price** (EUR/MWh): total cost divided by the energy consumed, where the energy is the active node-hours times $P_{\text{node}}/1000$. It is the average price the policy paid per megawatt-hour of active computation and does not depend on how much work was completed. It is the primary cost metric.
- **Completion rate**: C/J.
- **QoS violations** and **violation rate**: the number of jobs that missed their completion deadline (Section 3.5) and that number as a percentage of J. The violation rate is also called the deadline-miss rate, or miss rate for short. Differences between violation rates are given in percentage points. For example, 9.6 points more means that 9.6 more of every 100 submitted jobs miss their deadline.
- **Waiting time** (h): time from submission to first start, reported as mean and 95th percentile over completed jobs.
- **Response time** (h): time from submission to completion, mean over completed jobs.
- **Slowdown**: response time divided by runtime, mean over completed jobs. With runtimes of at least one hour this equals the bounded slowdown of the scheduling literature.
- **Reserved utilization**: reserved node-hours divided by $N \cdot T$. For P3R, paused jobs count as reserved.
- **Suspension**: the number of jobs paused at least once after running, the total suspended job-hours, and the suspended reserved node-hours.

The report script also ranks the λ values by a penalized objective, total cost plus 1000 EUR per QoS violation. This is one possible operational preference, used to pick a single representative λ per load for the figures, and not a claim about the economic value of a missed deadline.

Two reading rules apply throughout. Total cost is never interpreted without the completed work next to it, because a policy that completes fewer jobs pays less for that reason alone. The normalized paid price is the metric that is free of this problem, and the results begin with it.

5.4 Does P3R Shift Work to Cheaper Hours?

The answer to Question 1 is yes when the cluster has spare capacity and no when it is overloaded. All comparisons in this and the following sections are paired. Each value of P3R is compared with EASY on the identical workload and seed, and the confidence intervals are those of the per-seed differences. Table 5.3 lists the underlying means for the full-year experiment. On average 1392 jobs are submitted at low load, 3073 at medium, 5048 at high, and 3955 at mixed load, and the completed jobs in the table are counted against these numbers.

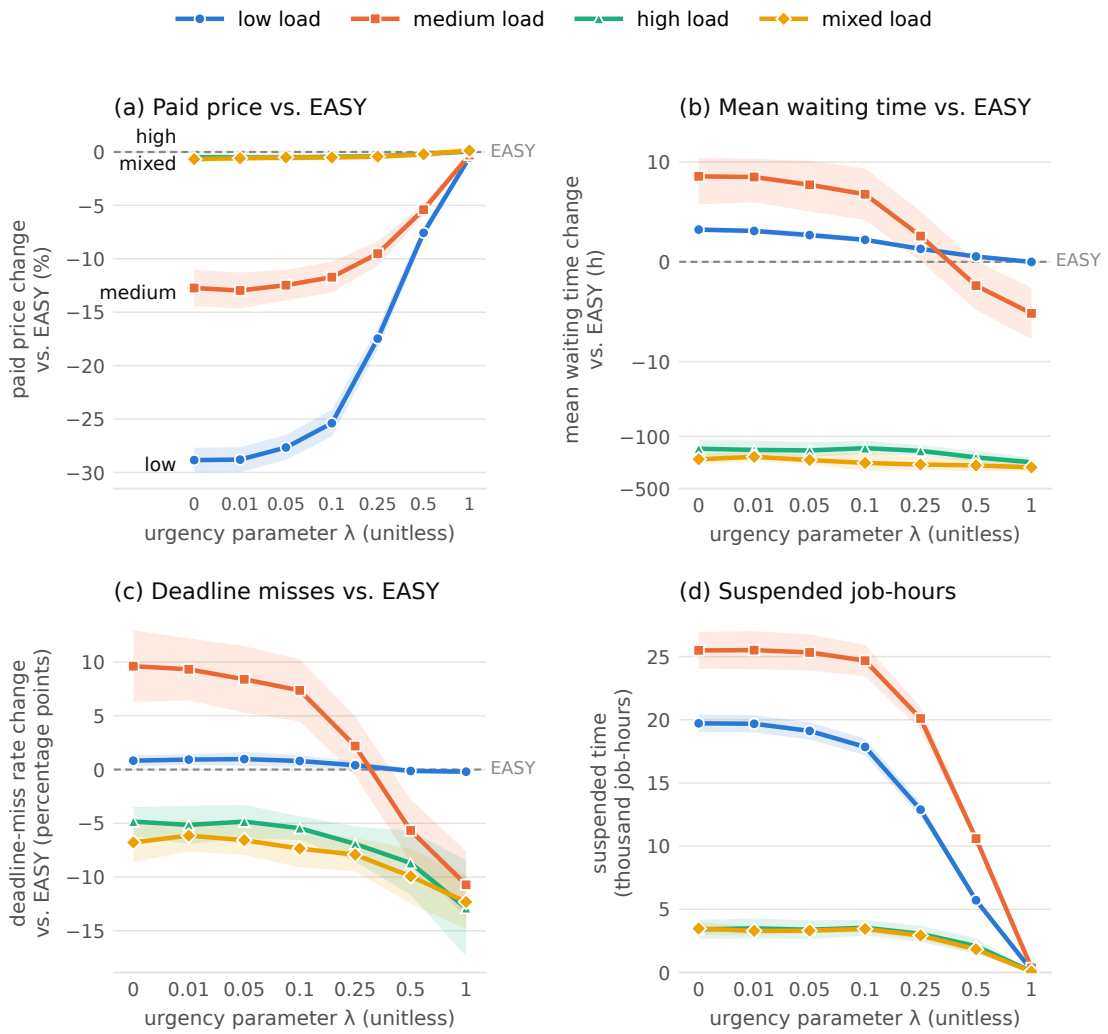


Figure 5.2: *Effect of the urgency parameter λ over the full year, paired against EASY backfilling on identical workloads. Points are means over ten seeds, shaded bands are 95 % confidence intervals of the per-seed differences, and the dashed line marks EASY. Panel (b) is linear within $\pm 10\,h$ and logarithmic beyond, so that the overload levels fit on the same axis.*

At $\lambda = 0$ P3R pays $28.8 \pm 1.2\,\%$ less per megawatt-hour of active computation than EASY at low load and $12.7 \pm 1.7\,\%$ less at medium load (Figure 5.2a). At high and mixed load the reduction is $0.5 \pm 0.4\,\%$ and $0.7 \pm 0.3\,\%$, barely distinguishable from zero. In absolute terms, EASY and FCFS pay about 79 EUR/MWh at every load. The two baselines are price-blind and pay close to the mean of the 2024 trace (78.5 EUR/MWh), while P3R pays 56.5 EUR/MWh at low load and 69 EUR/MWh at medium load.

Two features of the λ curve matter for the rest of the evaluation. First, λ does little below 0.1. The three lowest values are within about one percentage point of each other, because the penalty of Equation (4.9) is small against the price differences that drive the decision. The parameter therefore takes effect only from 0.1 upward. Second, at $\lambda = 1$ the reduction shrinks to $0.5 \pm 0.1\,\%$ at low load and $0.3 \pm 0.2\,\%$ at medium load.

Table 5.3: *Full year 2024: means over ten seeds with 95 % confidence intervals. Mean resp. is the mean response time from submission to completion, which includes the hours a job spends paused. The settings* $\lambda = 0.01$ *and* 0.05 *are omitted; Figure 5.2 includes them.*

Load	Policy	Paid price (EUR/MWh)	Completed (jobs)	QoS viol. (%)	Mean wait (h)	p95 wait (h)	Mean resp. (h)	Suspended (job-h)
low	FCFS	79.3 ± 1.1	1387 ± 21	0.7 ± 0.4	0.2 ± 0.2	0 ± 0	24.5 ± 0.6	0 ± 0
	EASY	79.3 ± 1.1	1387 ± 21	0.6 ± 0.3	0.1 ± 0.1	0 ± 0	24.4 ± 0.6	0 ± 0
	$\lambda = 0$	56.5 ± 1.3	1387 ± 21	1.4 ± 0.5	3.3 ± 0.3	13 ± 1	41.9 ± 1.2	19718 ± 678
	$\lambda = 0.1$	59.2 ± 1.3	1387 ± 21	1.4 ± 0.6	2.3 ± 0.3	10 ± 1	39.5 ± 1.2	17853 ± 660
	$\lambda = 0.25$	65.5 ± 1.3	1387 ± 21	1.0 ± 0.5	1.4 ± 0.2	7 ± 0	35.0 ± 0.9	12884 ± 548
	$\lambda = 0.5$	73.3 ± 1.2	1387 ± 21	0.5 ± 0.1	0.6 ± 0.1	4 ± 0	29.1 ± 0.6	5718 ± 213
	$\lambda = 1$	78.9 ± 1.1	1387 ± 21	0.4 ± 0.1	0.1 ± 0.0	0 ± 0	24.5 ± 0.6	191 ± 16
medium	FCFS	79.0 ± 0.6	3061 ± 40	26.3 ± 7.0	17.9 ± 7.4	94 ± 36	43.0 ± 7.7	0 ± 0
	EASY	79.0 ± 0.6	3065 ± 41	16.5 ± 4.7	8.5 ± 3.4	52 ± 20	33.7 ± 3.7	0 ± 0
	$\lambda = 0$	68.9 ± 1.3	3060 ± 40	26.1 ± 2.5	17.1 ± 2.1	77 ± 13	50.4 ± 2.0	25491 ± 1453
	$\lambda = 0.1$	69.7 ± 1.1	3059 ± 41	23.9 ± 2.6	15.3 ± 2.1	70 ± 11	48.4 ± 2.1	24680 ± 1241
	$\lambda = 0.25$	71.5 ± 1.0	3061 ± 41	18.7 ± 2.2	11.1 ± 1.7	52 ± 9	42.7 ± 1.8	20101 ± 898
	$\lambda = 0.5$	74.7 ± 0.7	3062 ± 41	10.8 ± 1.9	6.1 ± 1.3	30 ± 6	34.7 ± 1.5	10596 ± 329
	$\lambda = 1$	78.8 ± 0.6	3065 ± 41	5.8 ± 1.6	3.3 ± 1.0	17 ± 6	28.6 ± 1.3	367 ± 39
high	FCFS	78.6 ± 0.2	3978 ± 177	97.2 ± 0.7	853.4 ± 144.4	1715 ± 271	878.1 ± 144.9	0 ± 0
	EASY	78.7 ± 0.2	4156 ± 176	95.9 ± 1.3	685.1 ± 140.1	1457 ± 279	709.7 ± 140.6	0 ± 0
	$\lambda = 0$	78.3 ± 0.2	4251 ± 152	91.1 ± 2.6	538.9 ± 116.8	1268 ± 250	563.6 ± 117.1	3412 ± 751
	$\lambda = 0.1$	78.3 ± 0.1	4230 ± 153	90.5 ± 2.2	541.5 ± 117.2	1272 ± 246	566.3 ± 117.5	3540 ± 620
	$\lambda = 0.25$	78.3 ± 0.2	4263 ± 146	89.0 ± 2.7	529.0 ± 118.8	1244 ± 240	553.6 ± 119.1	3059 ± 634
	$\lambda = 0.5$	78.5 ± 0.2	4273 ± 144	87.2 ± 4.0	494.6 ± 115.7	1234 ± 239	519.1 ± 116.0	2090 ± 634
	$\lambda = 1$	78.7 ± 0.1	4331 ± 169	83.0 ± 5.6	463.4 ± 126.8	1164 ± 273	487.2 ± 127.2	80 ± 29
mixed	FCFS	78.6 ± 0.2	3061 ± 72	97.6 ± 0.7	973.4 ± 104.0	1895 ± 184	1003.8 ± 104.5	0 ± 0
	EASY	78.7 ± 0.2	3238 ± 75	96.5 ± 0.6	751.9 ± 101.8	1573 ± 160	781.7 ± 102.3	0 ± 0
	$\lambda = 0$	78.2 ± 0.3	3344 ± 80	89.8 ± 1.8	549.6 ± 93.4	1397 ± 198	579.5 ± 93.9	3477 ± 501
	$\lambda = 0.1$	78.3 ± 0.2	3330 ± 79	89.2 ± 1.6	525.6 ± 93.9	1397 ± 179	555.6 ± 94.4	3450 ± 280
	$\lambda = 0.25$	78.4 ± 0.3	3337 ± 72	88.6 ± 1.5	514.4 ± 86.6	1363 ± 181	544.2 ± 86.9	2928 ± 305
	$\lambda = 0.5$	78.5 ± 0.2	3360 ± 83	86.6 ± 2.4	507.4 ± 81.2	1342 ± 197	536.8 ± 81.6	1838 ± 289
	$\lambda = 1$	78.8 ± 0.2	3381 ± 104	84.2 ± 2.3	492.4 ± 85.9	1282 ± 198	521.3 ± 86.4	59 ± 18

P3R has become a run-when-possible scheduler that still keeps reservations for paused jobs, and it pays almost what EASY pays.

Total cost follows the paid price closely, because all policies execute nearly the same work. Over the year P3R pays 8362 against EASY's 11,745 EUR at low load, which is 28.8 % less for the same energy. At medium load it pays 23,060 against 26,622 EUR, or 13.4 % less, with 0.8 % less energy. The paid price remains the primary metric because it is independent of the small differences in executed work.

Under overload the saving disappears, and the queues explain why. The high and mixed levels offer 1.23 times the cluster's capacity (Section 5.2), so no policy can keep up and every queue grows for the whole year. At high load EASY's mean waiting time is 685 h and its violation rate 96 %, and FCFS waits 853 h. P3R waits 539 h at $\lambda = 0$ and 463 h at $\lambda = 1$, with 91 % and 83 % violations, so it degrades least, for a reason discussed in Section 5.5. What it cannot do is shift work. With every node held by a running or paused job, at a reserved utilization of 99 %, Question 3 of Section 4.4 decides when a job runs, not Question 2. Queueing consumes the slack, the urgency test forces the job to run, and price shifting has nothing to shift into. The half a percent that remains is what P3R gains from choosing among the few jobs that ever have a choice. The rest of the evaluation therefore concentrates on the low and medium levels,

where the cluster has room.

5.5 What Does It Cost Users?

The saving is paid for in waiting and interruption at both loads, and at medium load also in missed deadlines. Against EASY, the most price-seeking setting raises the deadline-miss rate by less than one percentage point at low load and by almost ten at medium load. These costs fall quickly with λ, and at medium load $\lambda = 0.5$ avoids them relative to EASY.

The added waiting appears at both loads. At low load, at $\lambda = 0$, the mean waiting time rises from EASY's 0.1 h to 3.3 h, an increase of 3.2 ± 0.3 h (Figure 5.2b), and the 95th percentile from 0 to 13 h. The mean slowdown, response time over runtime, rises from 1.0 to 2.0. Short jobs feel this most, since an hour of waiting is a large fraction of a two-hour job and a small fraction of a two-week job. Because paused jobs also finish later, the mean response time grows more than the waiting, from 24.4 to 41.9 h, an increase of 17.5 ± 0.7 h. At medium load EASY itself waits 8.5 h on average, with a 95th percentile of 52 h. P3R at $\lambda = 0$ waits 17.1 h, which is 8.6 ± 2.8 h more, with a 95th percentile of 77 h and a slowdown of 2.7 against EASY's 2.3. The added waiting falls with λ. At $\lambda = 0.25$ it is 2.6 ± 2.5 h and not distinguishable from zero, at $\lambda = 0.5$ it waits 2.4 h less than EASY, again within the noise, and only at $\lambda = 1$ does it wait clearly less, by 5.2 h. The mean response time grows by 16.8 ± 2.9 h at $\lambda = 0$ and by 9.0 ± 2.6 h at $\lambda = 0.25$, while at $\lambda = 0.5$ the increase of 1.0 ± 2.5 h is not distinguishable from zero.

Interruption is a cost at both loads as well. At $\lambda = 0$, 61 % of all jobs at low load are paused at least once after starting (853 of 1392), and the suspended time sums to 19,700 job-hours over the year, about 23 hours per paused job. At medium load 1402 jobs, or 46 %, are paused at least once, for 25,500 job-hours in total (Figure 5.2d). Every suspended job-hour is a reservation held without computing (Assumption 6), which is why the reserved utilization of P3R is 51 % at low load, where EASY's is 33 %, and 93 % at medium load, where EASY's is 76 %. These interruptions are free in the simulator. In a real system each one is a checkpoint and a restart, and the count of paused jobs is the number that the checkpoint overhead of Section 7.3 would multiply. At $\lambda = 1$ interruptions become rare. Only 64 jobs at low load and 125 at medium load are paused at least once, for 191 and 367 suspended job-hours.

Deadline misses, finally, rise at both loads, slightly at low load and clearly at medium load. At low load the violation rate at $\lambda = 0$ is 1.4 % against EASY's 0.6 %, a difference of 0.83 ± 0.51 points. At $\lambda = 0.1$ and 0.25 the difference is 0.80 ± 0.54 and 0.40 ± 0.39 points, and from $\lambda = 0.5$ on the rate is at or below EASY's (Figure 5.2c). At medium load the most price-seeking setting misses 26.1 % of deadlines against EASY's 16.5 %, a miss rate 9.6 ± 3.3 points *higher* than EASY's. The difference falls with λ to 2.2 ± 2.8 points higher at $\lambda = 0.25$, which is not distinguishable from zero, and turns into a miss rate 5.7 ± 2.9 points *lower* than EASY's at $\lambda = 0.5$ and 10.7 ± 3.1 points lower at $\lambda = 1$. Under overload P3R misses 91 % of deadlines at $\lambda = 0$ and 83 % at $\lambda = 1$ at high load, where EASY misses 96 %.

The advantage over EASY at large λ has to be attributed correctly. It is largest

at $\lambda = 1$, where P3R does no price seeking at all, so it does not come from price awareness. It does not come from pausing either, because a paused job keeps its nodes (Assumption 6) and so never makes room for another job. Two other differences from EASY remain. P3R admits waiting jobs in order of deadline and slack, where EASY keeps arrival order, and it starts any job that fits, where EASY holds a reservation for the blocked head of the queue and starts a later job only if that job does not delay the head (Sections 4.3 and 4.4.4). The fourth robustness check (Section 5.7) separates the two by running P3R with arrival order instead of deadline order. At $\lambda = 1$ and medium load, the admission rule accounts for 6.3 ± 2.1 of the 10.7 points and the deadline order for 4.4 ± 1.3. Under high and mixed load the deadline order contributes more, about 8 of an advantage of 12 to 13 points.

The same check shows what the deadline order does for price shifting. The price saving does not depend on it. With arrival order P3R at $\lambda = 0$ pays 28.9 % less than EASY at low load and 13.0 % less at medium load, against 28.8 and 12.7 % with deadline order. The deadline cost of the saving does depend on it. With arrival order the fully price-seeking setting has a miss rate 20.6 ± 2.7 points higher than EASY's at medium load, against 9.6 ± 3.3 points higher with deadline order. Relative to the same scheduler without price seeking, which is P3R at $\lambda = 1$, the setting $\lambda = 0$ has a miss rate 1.0 ± 0.4 points higher at low load and 20.3 ± 1.7 points higher at medium load. The deadline order thus halves the deadline cost of price shifting against EASY, but it does not remove it.

5.6 When Does It Help or Fail?

Figure 5.3 puts the saving and its cost in waiting together, with one panel per load. Moving along a curve from $\lambda = 0$ toward $\lambda = 1$ trades price for waiting, and the star marks where EASY sits. At low load the curve descends steeply to the right of the star, so P3R buys its saving with waiting. At medium load the most price-seeking settings lie below and to the right of the star. They pay less but wait longer, by 8.6 ± 2.8 h at $\lambda = 0$. At $\lambda = 0.25$ the curve passes just below and to the right of the star, with a 9.5 % lower price and 2.6 ± 2.5 h more waiting, and from $\lambda = 0.5$ upward the curve lies to the left of the star, clearly so only at $\lambda = 1$. The choice of λ therefore decides how the gain splits between a lower price and shorter waits.

At high and mixed load the curves sit between about 460 and 560 hours of mean waiting, far from any acceptable service, with EASY at about seven hundred and FCFS at about nine hundred. At every load FCFS pays about the same price as EASY, waits longer, and misses more deadlines, and it serves only to confirm that the simulator reproduces the textbook relation between the two baselines. Three conclusions follow for the loads with room.

First, the useful range of λ is 0.1 to 0.5. Below 0.1 little changes, and at 1 P3R has converged to EASY's price. In between, the low-load curve has a knee. At $\lambda = 0.1$ P3R keeps 88 % of the maximum saving (25.4 of 28.8 %) for 68 % of the added waiting, at $\lambda = 0.25$ it keeps 61 % for 40 % of the waiting and 65 % of the suspended hours, and at $\lambda = 0.5$ it keeps 26 % for 17 % of the waiting. At medium load, $\lambda = 0.25$ keeps 75 % of the saving, 9.5 of 12.7 %. Its jobs start about as early as under EASY and miss about as

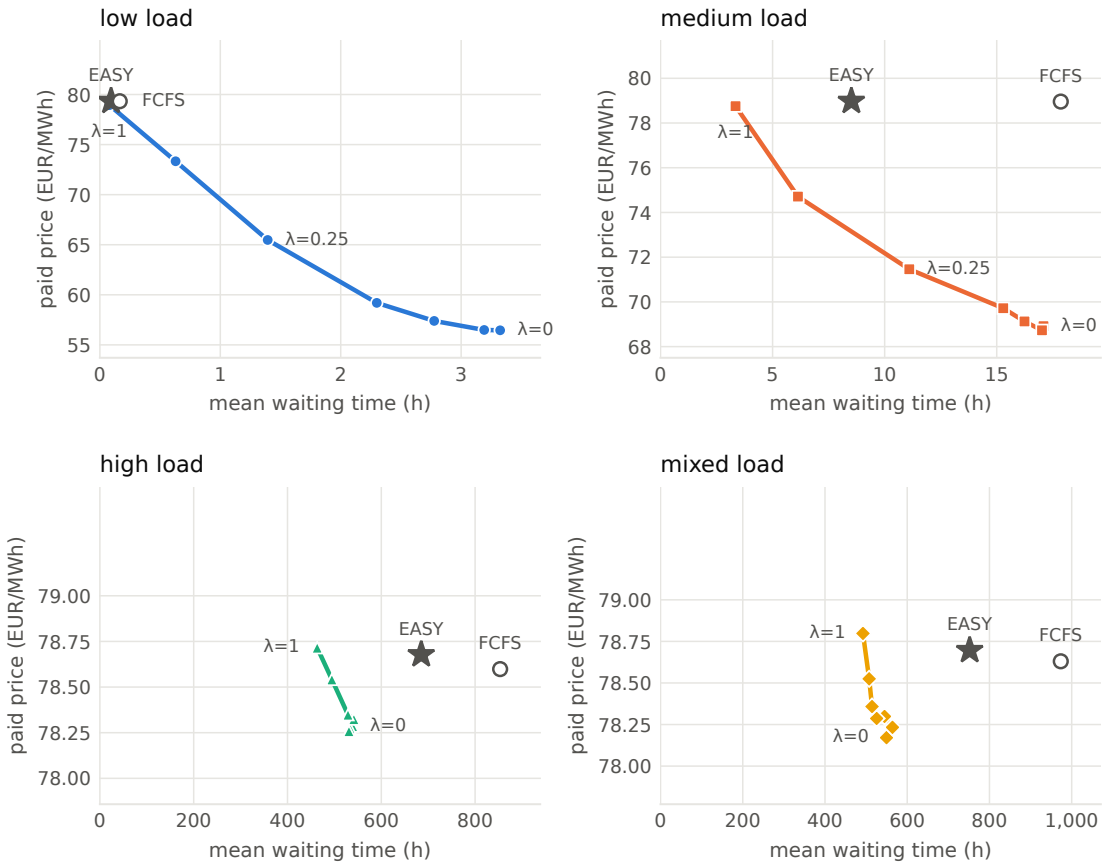


Figure 5.3: *The cost/QoS trade-off over the full year, one panel per load level. Each curve traces P3R from $\lambda = 0$ to $\lambda = 1$; the star is EASY and the open circle FCFS on the same workloads. Values are means over ten seeds. The axes differ between panels, so compare positions within a panel.*

many deadlines, but they finish 9.0 ± 2.6 h later because of their pauses. $\lambda = 0.5$ keeps 42 % of the saving with no measurable delay in completion and a miss rate 5.7 points lower than EASY's. Relative to the same scheduler without price seeking, at $\lambda = 1$, every price-seeking setting still has a higher miss rate, 5.1 points higher at $\lambda = 0.5$ and 12.9 points higher at $\lambda = 0.25$. Which point an operator prefers depends on how an hour of waiting, a missed deadline, or an interruption is valued against a percent of electricity cost, which the simulator cannot decide.

Second, P3R pays off wherever the cluster has room, and what users pay for the saving changes with load. At low load they pay in waiting. P3R takes more than a quarter off the price for three hours of mean waiting, a tail that grows from zero to thirteen hours, an interruption for most jobs, and a miss rate less than one point higher than EASY's. At medium load the fully price-seeking setting takes an eighth off the price but adds about nine hours of waiting and raises the miss rate ten points above EASY's, and it pauses 1402 jobs. A moderate setting changes this. At $\lambda = 0.5$ two fifths of the saving remain, jobs finish as early as under EASY, the miss rate is lower than EASY's, and only the interruptions remain. Beyond capacity P3R buys nothing, because price shifting needs free capacity and slack at the same time, and overload

takes away both. A cluster that runs near saturation gains nothing from this policy, while a cluster with slack, or a partition reserved for checkpointable batch work, gains the most.

Third, the ranking of the λ values depends on the price of a violation. The report script ranks them by total cost plus 1000 EUR per violation and picks $\lambda = 1$ at every load. It does so because at 1 kW per node the electricity cost of a year's workload is 8 to 35 kEUR, while the violation term is a half to two and a half times as large at low load and seven to thirty-five times as large at medium load, depending on λ. With a violation priced at 100 EUR the ranking picks $\lambda = 0$ at low load, and at 200 and 300 EUR it picks $\lambda = 0.5$ there. At medium load and above it picks $\lambda = 1$ at every price tested. At 1 kW per node the simulated cluster draws about as much as the estimate for the AIP cluster in Section 6.2, so the cost term is of realistic size. A larger facility would shift the ranking toward lower λ. The ranking is one operational preference, and Chapter 6 discusses what the numbers mean at the scale of the motivating cluster.

5.7 Robustness

Five checks test whether the full-year results depend on the season, on the length of the horizon, on the particular workloads drawn, on the order in which P3R admits jobs, or on P3R knowing the job profiles.

The first check shows that the season changes what each pause is worth. Two 60-day windows with identical seeds, and therefore identical workloads, were run against different parts of the price trace. The winter window covers January to February, the months with the smallest daily spreads, and the summer window covers July to August, the months with the largest (Section 5.2). Appendix B lists the means of both windows. Figure 5.4 shows the paired price reduction in both windows and over the full year. At $\lambda = 0$ the saving at low load is 17.9 ± 1.8 % in winter and 33.7 ± 2.0 % in summer, and at medium load it is 11.5 ± 1.5 % and 19.0 ± 4.6 %. The full-year values lie between the two, closer to the summer value at low load and to the winter value at medium load.

What P3R does, by contrast, is the same in both windows. At low load the added mean waiting is 3.5 and 2.9 h, the suspended job-hours 3000 and 3200, and the violation rates differ from EASY's by 1.0 ± 1.4 points in both, neither distinguishable from zero. This behavior is set by the relative structure of the prices within a window, while the saving scales with the spread. The knee of the λ curve moves slightly. At $\lambda = 0.25$ P3R keeps 67 % of the maximum saving in winter and 62 % in summer. The penalty of Equation (4.9) scales with the price span, so the shift comes from the shape of the price curves in the two windows, not from their level.

The second check shows that the length of the horizon matters at high and mixed load, where the 60-day windows tell a different story than the full year does. In the windows the paired reduction is 5 to 7 %, against 0.5 to 0.7 % over the year, and Figure 5.5 shows the winter window, which gives the lower end of that range. The gap to the full year comes from the backlog of Section 5.4. After 60 days at 1.23 times capacity, EASY's mean waiting time at high load is 94 h and P3R's 39 h, and after a year they are 685 and 539 h. A short window catches the overload while the queues are

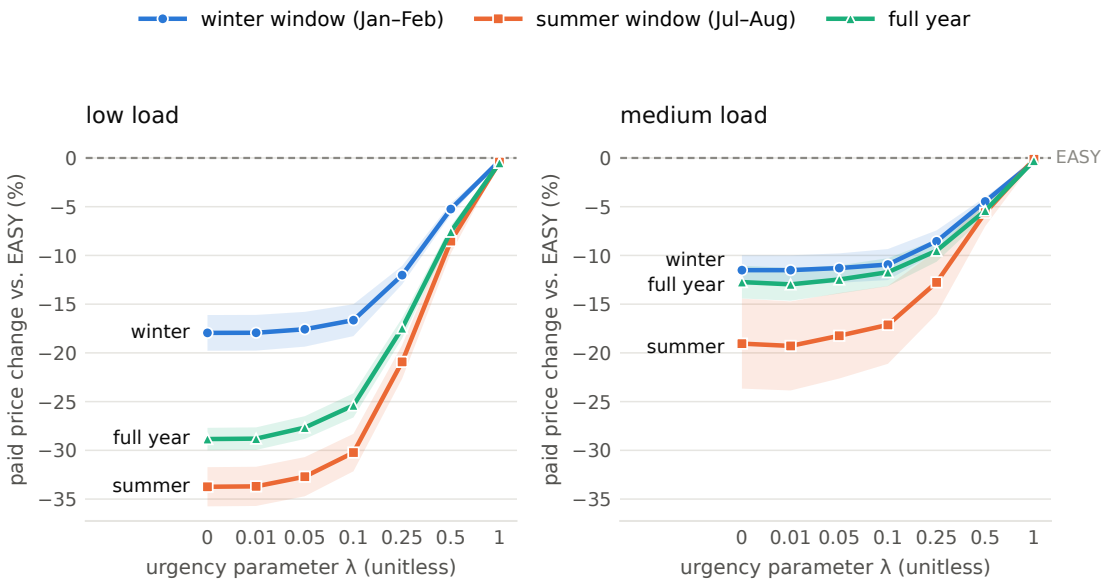


Figure 5.4: *Paired price reduction against EASY for the winter window, the summer window, and the full year, at low and medium load. Shaded bands are 95 % confidence intervals over ten seeds.*

still filling and P3R still has choices, and so it overstates what P3R can do at saturation. The low- and medium-load results do not show this horizon dependence, which is consistent with their bounded queues. A horizon of at least several months is therefore necessary before any result at or above capacity can be read.

The third check repeats the winter window with a second set of ten seeds, 42 to 51, which were used during development. The run reproduces P3R's paid price within the confidence intervals reported here, with 58.5 against 58.6 EUR/MWh at low load and 62.1 against 61.9 EUR/MWh at medium load for $\lambda = 0$. The paired reduction against EASY is 17.2 ± 1.4 % against 17.9 ± 1.8 % at low load and 10.6 ± 2.0 % against 11.5 ± 1.5 % at medium load. The effect does not depend on the particular workloads drawn.

The fourth check varies the order in which P3R admits waiting jobs (Section 4.4.4) on the workloads and prices of the full-year experiment. Breaking ties between equal deadlines the other way round, by higher current-step cost and then higher job identifier with slack ignored, changes the paid price by at most 0.2 % and the violation rate by at most 0.7 points at any load, at $\lambda = 0$ and at $\lambda = 1$, and none of these differences is distinguishable from zero. The tie-breaking rule therefore does not affect the results. Replacing the deadline order by plain arrival order, which also drops the rule that mandatory jobs go first, matters more. It leaves the price saving unchanged but raises the violation rate. At $\lambda = 1$ the rate rises from 5.8 to 10.2 % at medium load, where EASY misses 16.5 %, and from 83.0 and 84.2 % to 91.5 and 91.8 % under high and mixed load. Section 5.5 uses these runs to attribute the deadline advantage over EASY.

The fifth check asks how much of the saving comes from knowing the job profiles. It reruns P3R on the workloads and prices of the full-year experiment with a planner

that treats every job as flat, assuming that all n_j reserved nodes are active in every remaining hour, while the cost is still charged for the true profile. At $\lambda = 0$ this profile-blind planner pays 27.5 % less than EASY at low load, against 28.8 % with the profile, a difference of 1.3 ± 0.1 points. At medium load the difference is 0.5 ± 0.5 points, at the limit of what ten seeds can distinguish from zero. The violation rate and the suspended hours change only slightly. At $\lambda = 0.5$ the profile adds 0.6 ± 0.2 points at low load and 0.5 ± 0.2 points at medium load, but there the blind planner also scales the urgency penalty by n_j instead of the mean active count, so $\lambda = 0$ is the cleaner comparison. Most of the saving therefore comes from shifting work into cheap hours at all. Knowing which hours of a job are active adds a small but measurable part at low load, where free capacity leaves room to place single steps.

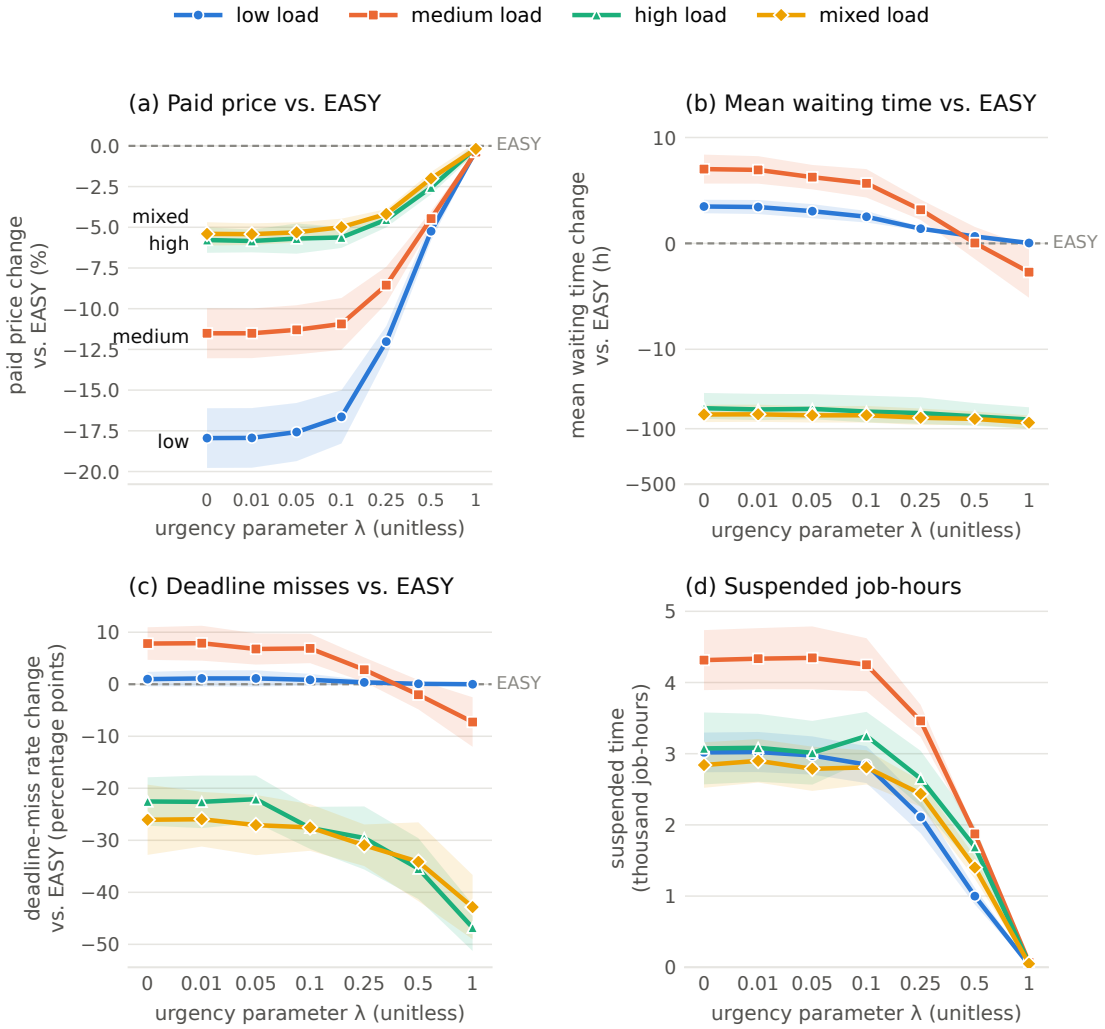


Figure 5.5: *Effect of the urgency parameter λ in the 60-day winter window, January to February 2024, at low, medium, high, and mixed load, paired against EASY backfilling on identical workloads. Points are means over ten seeds, shaded bands are 95 % confidence intervals of the per-seed differences, and the dashed line marks EASY. Panel (b) is linear within ±10 h and logarithmic beyond. The full-year sweep is Figure 5.2.*

Chapter 6

Discussion

This chapter interprets the results of the evaluation, estimates what they would mean for the AIP cluster, and discusses the limits that the model's assumptions set on them.

6.1 Interpretation

The research question of Chapter 1 asked whether knowing both the price signal and the job's activity lets a scheduler buy computation more cheaply without unacceptable service degradation. The evaluation gives a conditional answer, and the conditions are the more useful part of it.

The saving comes from shifting work into cheaper hours, which is possible for a job only when three things are present at once: a price difference within its window, slack to move it, and free capacity to hold it. The job's profile decides which of its hours are worth the most to move. The experiments vary each of these. The price difference is set by the season, and on identical workloads the saving at low load is 18 % in a winter window and 34 % in a summer window (Section 5.7), because the summer valleys of Figure 5.1 are deeper than the winter ones. The profile matters least. A planner that treats every job as flat keeps all but 1.3 points of the saving at low load and all but half a point at medium load (Section 5.7), so most of the saving comes from moving hours at all, not from choosing which steps to move. Slack is set by the class-dependent wait factor, and long jobs, with days of absolute slack, are the ones that can be moved across a whole price cycle. Capacity is set by the load, and it is the binding condition. The saving falls from 29 % to 13 % to under 1 % as the offered load rises from 0.34 to 0.77 to 1.23 of capacity, because a job that has waited for a node has no slack left to wait for a price.

The load also decides what the saving costs. At low load the cost is waiting. The cluster is two thirds idle and EASY starts almost everything at once, so any hour P3R holds a job is an hour the job would otherwise have been running. At medium load the fully price-seeking setting costs waiting, interruption, and deadlines at once. EASY waits 8.5 h on average there, and P3R at $\lambda = 0$ doubles that and has a miss rate ten percentage points higher. Its admission rule, which starts any job that fits in deadline order instead of holding nodes free for a blocked head (Section 5.5), recovers part of this but not all of it. In addition, almost half the jobs are paused at least once. In the model a pause costs nothing, but in a real system each pause requires a checkpoint and

a restart. The reservation rule (Assumption 6) is what limits both regimes. A paused job holds its nodes, so at medium load P3R runs at 93 % reserved utilization to deliver the same work EASY delivers at 76 %. The nodes held by paused jobs are missing for the queue, which is the likely cause of much of the added waiting and of the missed deadlines, although no experiment here isolates it, and beyond capacity that headroom is gone. Releasing paused reservations, the extension listed in Section 7.3, would attack exactly this and is the change most likely to move the medium-load curve.

Which jobs should be eligible for shifting, and so bear the waiting and the interruptions, is a question the experiments do not answer directly. They report slowdown as a mean over all completed jobs and do not break it down by class, so the following is an argument from arithmetic rather than a measured result. A two-hour job paused for five hours has more than tripled its response time to save a few cents. A two-week job paused for the same five hours has not noticed. The profile lookahead also has more to work with on a long job, whose window spans several price cycles. A deployment would therefore restrict shifting to jobs above a runtime threshold and with a wait tolerance of at least a day, and leave short jobs to the baseline scheduler. Measuring the effect of this restriction is left to future work.

A deployment would also have to set λ, and how conservative to be depends on the load. At low load there is little reason to be conservative. With $\lambda = 0.1$ P3R keeps 88 % of the saving, adds 2.2 hours of mean waiting, and has a miss rate 0.80 ± 0.54 points higher than EASY's. At medium load the choice matters more. The fully price-seeking setting has a miss rate 9.6 points higher than EASY's. $\lambda = 0.25$ keeps three quarters of the saving and has about EASY's miss rate, but its jobs finish nine hours later. $\lambda = 0.5$ keeps 42 % of the saving, removes 58 % of the suspended hours, finishes jobs as early as EASY, and has a miss rate 5.7 points lower than EASY's. Against EASY, $\lambda = 0.5$ is therefore the safe setting at medium load, and $\lambda = 0.25$ the one for users who accept later results. Against the same scheduler without price seeking, both still have a higher miss rate (Section 5.6). Beyond capacity price shifting is inert, but the way P3R admits jobs is not. Under the same overload its violation rate lies 5 to 13 points below EASY's, depending on load and λ. That benefit needs no price signal. It comes mostly from the deadline order (Section 5.5), not from price awareness.

For the price saving itself, the closest prior work is Kammeyer's start-time scheduler (Section 2.5), which delays job starts but never pauses a running job. Against unmodified backfilling, Kammeyer reports energy-cost savings of up to about a third at moderate load, 4 to 8 % at high load, and almost none under extreme load [11, Sec. 6.5]. The pattern matches the 29 % and 13 % at low and medium load and the near-zero saving beyond capacity found here, but the numbers themselves are not comparable. Kammeyer measures the total energy cost including idle nodes, on 2023 prices, with a node power model and load levels of his own definition, and his largest saving is the best case over start dates and lookahead windows. This thesis measures the price paid per active node-hour on 2024 prices, averaged over ten seeds. What the comparison does show is that the overload limit belongs to neither mechanism. A later start and a pause both need free capacity to move work into, and neither finds any once the queue exceeds the cluster. The mechanisms differ in what the saving costs. A later start costs only waiting, while a pause in a real system also costs a checkpoint and a restart.

6.2 Savings at the Scale of the AIP Cluster

This section estimates what the measured percentages would mean for the ∼6,600-core cluster at AIP introduced in Section 1.1. All costs in Chapter 5 come from a 64-node cluster with a nominal 1 kW per node, and none of the AIP cluster's power draw, utilization, tariff, or checkpointing data was available, so the estimate is a range built from stated assumptions.

The yearly saving is

$$\text{saving per year} = P_{\text{full}} \cdot u \cdot T \cdot \bar{p} \cdot r \cdot c, \tag{6.1}$$

with the following quantities.

1. *Full-load power* P_{full}: the active power of the whole cluster at full load. Table 6.1 evaluates 5, 8, and 12 W per core, which covers lean and dense compute nodes. The AIP cluster mixes Xeon Gold 6252, Xeon Platinum 8352Y, and Xeon 6960P processors, whose rated power per core is 6.3, 6.4, and 6.9 W [10]. Memory, network, and power supply add to this, so 8 W per core is the closest of the three values.

2. *Active utilization* u: EASY's active node-hours divided by the capacity, as measured in the full-year experiment, $u = 0.26$ at low load and $u = 0.60$ at medium load.

3. *Hours per year* T: 8784 h, the length of 2024.

4. *Paid price* $\bar{p}$: the price EASY pays per megawatt-hour in the full-year experiment, 79.3 EUR/MWh at low load and 79.0 EUR/MWh at medium load, so that r is applied to the price it was measured against.

5. *Price reduction by P3R* r: how much less P3R at $\lambda = 0$ pays per megawatt-hour of active computation than EASY on the same workloads, as a share of EASY's price and averaged over the ten seeds of the full-year experiment (Section 5.4). It is 29 % at low load and 13 % at medium load.

6. *Checkpointable share* c: the share of the workload that can be checkpointed. Table 6.1 evaluates half and all of it.

In absolute terms the numbers are small. At wholesale prices, the active computation of such a cluster costs 6 to 33 kEUR per year, and P3R would save 0.9 to 4.2 kEUR of it. The percentages from Chapter 5 still hold, but they apply to a small base. Grid fees, levies, and taxes make up much of a German electricity bill [2] and do not change by the hour, and active computation is only part of what a cluster draws.

The saving grows in two ways. It scales linearly with the size of the facility, so at the same percentages a data center with 10 MW of compute nodes would save roughly 0.25 to 0.5 million euro per year. P3R can also follow any hourly signal, not only price. With the carbon intensity of the grid as the signal, the gain would be counted in emissions rather than in euro.

Before any deployment, five inputs would have to be measured on the real cluster: the active and idle power per node, the utilization and its daily pattern, the tariff and

Table 6.1: *Estimated yearly saving for a 6,600-core cluster at the price EASY pays, for P3R at* $\lambda = 0$.

Power (W/core)	Full-load power (kW)	Active utilization	Price reduction	Checkpointable share	Active energy (MWh/yr)	Saving (kEUR/yr)
5	33	0.26	29%	100%	76	1.7
5	33	0.26	29%	50%	76	0.9
5	33	0.60	13%	100%	174	1.7
5	33	0.60	13%	50%	174	0.9
8	53	0.26	29%	100%	122	2.8
8	53	0.26	29%	50%	122	1.4
8	53	0.60	13%	100%	278	2.8
8	53	0.60	13%	50%	278	1.4
12	79	0.26	29%	100%	183	4.2
12	79	0.26	29%	50%	183	2.1
12	79	0.60	13%	100%	417	4.2
12	79	0.60	13%	50%	417	2.1

which part of it varies by the hour, the share of jobs that can checkpoint and what a checkpoint costs, and a few weeks of accounting data to replace the synthetic workload. The simulator had to assume all five, and the assumed ranges alone already move the saving in Table 6.1 by almost a factor of five.

6.3 Limitations

The assumptions of Section 3.6 bound what the results can claim, and this section states the consequences, grouped by the kind of validity they threaten. Internal validity concerns whether the simulator measures what it claims, construct validity whether the metrics mean what the thesis says, and external validity whether the results transfer to a production system.

The main threat to internal validity is attribution. All policies share one QoS rule and one end-of-horizon rule (Section 3.5) and receive the same jobs, so their violation and completion counts are directly comparable, but P3R differs from EASY in three ways at once. It starts any job that fits instead of holding a reservation for the blocked head, it admits jobs in deadline order, and it pauses jobs to seek cheap hours. The price saving is cleanly attributable to the third difference, because it vanishes at $\lambda = 1$ while the other two remain, and because it does not change when the deadline order is replaced by arrival order (Section 5.7). The better deadline performance cannot be credited to price shifting, because it is largest at $\lambda = 1$, and it cannot be credited to pausing, because a paused job keeps its nodes. The same check splits it between the admission rule and the deadline order (Section 5.5). Relative to EASY, price shifting at $\lambda = 0$ raises the miss rate by about one percentage point at low load and ten at medium load, and by twice as much at medium load with arrival order instead of deadline order. Compared with the same policy without price seeking, at $\lambda = 1$, it adds misses and never removes any.

The end-of-horizon effect is a second threat to internal validity. Jobs submitted

near the end of the horizon cannot complete and count as violations under every policy. The effect is small over a year but weighs more in a 60-day window, which is why the windows show higher violation rates than the full year at low load. At medium load the windows show lower rates instead, because a 60-day queue has less time to build up than a year-long one. Finally, P3R breaks ties between jobs with equal deadlines by smaller slack, lower current-step cost, and lower job identifier (Section 4.4.4). This is one deterministic choice among several, but reversing it changes no result by more than 0.2 % in price or 0.7 points in violations, and none of these differences is distinguishable from zero (Section 5.7).

Construct validity is threatened by the cost metrics, in which the active-node count stands in for power. Real nodes draw power that depends on what they compute, and idle nodes still draw power. The active-node profile captures whether a node is working at all, not how hard, so the paid price is a price per active node-hour, not per joule. The cost model also excludes idle and base power, cooling, and infrastructure (Assumption 8), so the reported cost is the variable part of the bill and the percentage savings apply to that part only. The synthetic profiles are a further threat, because they are plausible shapes rather than measurements. The saving depends on how much of a job's activity can be moved. With flat profiles a pause still moves single hours, but every hour of a job then costs the same, so the planner can no longer put active steps into cheaper hours than quiet ones. The fifth robustness check bounds this effect: planning with flat profiles lowers the saving by 1.3 points at low load and by 0.5 points at medium load (Section 5.7).

The violation rate has a blind spot as well. It counts jobs, not node-hours, and none of the results is broken down by job width. Because P3R holds no reservation for a job that does not fit (Section 4.4.4), narrower jobs can overtake a wide job again and again. A separate run on the workloads and prices of the full-year experiment shows how strong this is at medium load, counting only jobs whose deadline lies inside the year. Jobs of the widest class, 10 nodes, wait 117 h on average under P3R at $\lambda = 0$ and 25 h at $\lambda = 1$, against 19 h under EASY, and 2 of the 1004 such jobs of all ten seeds never start within the year, against none under EASY. Jobs of 8 nodes wait 63 h at $\lambda = 0$ against 17 h. Jobs of one to four nodes wait close to what they wait under EASY, 4 to 18 h against 5 to 13 h. Part of the waiting that price shifting adds therefore falls on a few wide jobs. A deployment would need a rule that eventually reserves nodes for a long-waiting job, as EASY does for the head of its queue.

The first threat to external validity is that the workloads are synthetic. Their arrival rhythm, class mix, node requests, and wait tolerances are stated in Section 5.2 and are plausible, but no production log was replayed, so the absolute numbers describe this family of workloads rather than any site. The simulated cluster has 64 homogeneous nodes, while the motivating cluster has ∼6,600 cores and limits a single job to about 1000 cores, roughly a sixth of the cluster. The synthetic workloads follow this limit with at most 10 of the 64 nodes per job, but the mix of job widths below it is not measured. Only one price year, 2024, and one cluster size, 64 nodes, are simulated. The baselines are also simpler than the scheduler that the AIP cluster runs. The cluster orders its queue by fair share, which ranks jobs by their users' past usage (Assumption 9), while all three simulated policies ignore who submitted a job. Slurm's backfilling starts a lower-priority job only if that delays none of the higher-priority jobs, not only the first

one as in EASY [20]. The comparison with EASY therefore shows what P3R changes relative to a simple baseline, not relative to the scheduler in production, and on the real cluster P3R would have to be combined with the fair-share priority. The simulated scheduler knows the whole price trace (Assumption 3), whereas a deployed scheduler would know one day ahead with certainty and beyond that only a forecast. Every hour of delay beyond the known horizon adds forecast risk that the simulator does not see. In the model, every job can be checkpointed, and checkpointing costs neither time nor energy (Assumptions 4 and 5). In a real system the 61 % of jobs that P3R pauses at low load would each pay a checkpoint and a restart, and jobs that cannot be checkpointed at all would be excluded from shifting. Both reduce the saving, and the reported figures are the upper bound stated in Section 3.6.

Of all the assumptions, the reservation rule (Assumption 6) probably has the largest effect on the results, although no experiment here varies it. It is what turns every pause into withheld capacity and what makes the medium-load and overload curves flatten (Section 5.6). It is also the one on which a real resource manager behaves differently. In Slurm, a job that is stopped by a signal keeps its allocation and releases nothing, which is what the model assumes. A job suspended by the scheduler releases its processors for allocation to other jobs while it stays resident in memory, and a requeued job returns to the queue and is started again on whatever resources are free [16, 18]. Both give the capacity of a paused job back to the queue. The model covers only the first behavior, so the results describe the most conservative form of pausing. Under the other two, the capacity cost of pausing shrinks or disappears and the resume guarantee becomes the open problem, which is the extension discussed in Section 7.3.

Chapter 7

Conclusion and Future Work

A scheduler that knows the hourly electricity price and may pause jobs could make a cluster like AIP's pay less for the same computation, but only when the cluster has room to wait. In simulation, P3R pays more than a quarter less per megawatt-hour of active computation than EASY backfilling when the cluster is lightly loaded, an eighth less when it is busy, and almost nothing once more work arrives than the cluster can serve. On a busy cluster the saving has a price in waiting and missed deadlines, which a moderate setting of the urgency parameter avoids for part of the saving. Most of the saving comes from shifting work in time; knowing each job's activity profile adds only a small part. In euros the saving is small, and it rests on assumptions that still have to be checked on the real cluster.

7.1 Answers to the Research Questions

Q1 asked whether P3R shifts active work to cheaper hours. It does as long as the cluster has spare capacity. At $\lambda = 0$ it pays 29 % less per megawatt-hour than EASY at low load and 13 % less at medium load. Once the offered load exceeds capacity, the saving falls below 1 %, because a job that has waited for nodes has no slack left to wait for a price (Section 5.4). The season matters as well. On identical workloads, the saving at low load is 18 % in winter and 34 % in summer (Section 5.7).

Q2 asked what the shift costs users. At low load the cost is waiting and interruption. The mean wait rises from 0.1 to 3.3 hours, results arrive 17 hours later on average, 61 % of the jobs are paused at least once, and the miss rate rises from 0.6 to 1.4 %, 0.8 points above EASY's. At medium load the fully price-seeking setting costs more. The mean wait doubles from 8.5 to 17 hours, almost half of the jobs are paused at least once, and the miss rate rises from 16.5 to 26.1 %, 9.6 points above EASY's. These costs fall quickly with λ. At $\lambda = 0.5$ P3R keeps 42 % of the medium-load saving, its jobs finish as early as under EASY, and its miss rate is lower than EASY's. At $\lambda = 0.25$ it keeps three quarters of the saving, but jobs finish nine hours later (Section 5.5). That P3R has a lower miss rate than EASY at large λ comes from the way it admits jobs, in deadline order and without holding nodes for a blocked job, not from price awareness.

Q3 asked when the policy helps and when it fails. It helps when three things come together for a job: a price difference within its window, slack to move it, and free capacity to hold it. It fails when the last of these is missing, which is the case under

overload. Knowing the job's profile adds little on top. A planner that treats every job as flat keeps all but 1.3 points of the saving at low load and all but half a point at medium load (Section 5.7). Between these regimes, λ sets the trade-off. At low load $\lambda = 0.1$ keeps 88 % of the saving for 2.2 hours of additional waiting, and at medium load $\lambda = 0.5$ keeps 42 % without later completion and with a lower miss rate than EASY (Section 6.1).

All of these numbers are upper bounds. In the model, checkpointing costs neither time nor energy and the scheduler knows the prices of the whole year (Section 3.6).

7.2 Recommendations for the AIP Cluster

At the scale of the AIP cluster, the active computation costs about 6 to 33 kEUR per year at wholesale prices, and P3R would save 0.9 to 4.2 kEUR of it (Section 6.2). That alone does not justify checkpointing and pausing jobs, and it already assumes a tariff that passes hourly prices on. Price shifting becomes worthwhile when the goal is lower carbon emissions rather than lower cost. With the carbon intensity of the grid as the signal, P3R would move work into hours with more wind and solar power, and the gain would be counted in emissions instead of euros.

If the institute wants to try it, a trial should start conservatively, for example at $\lambda = 0.5$, only for long jobs that already checkpoint and whose users can wait at least a day, and then lower λ step by step while watching completion times and missed deadlines. P3R also needs two inputs that the cluster does not provide today: an activity profile for each job and a stated wait tolerance.

Two differences from the simulation limit how far the results carry over (Section 6.3). The AIP cluster orders its queue by fair share, so P3R would have to be combined with the fair-share priority, and its Slurm backfilling is more careful than EASY. The comparison with EASY therefore shows what P3R changes relative to a simple baseline, not relative to the scheduler that AIP runs today.

A scheduler can only buy electricity cheaply when the cluster has room to wait: free nodes, jobs with slack, and hours that differ in price. How often the AIP cluster has that room is the first thing to measure, and the simulator built for this thesis is the place to test the answer.

7.3 Future Work

The assumption list of Section 3.6 doubles as a roadmap. Each idealization marks a concrete extension. The following directions are ordered from model fidelity toward deployment.

- **Real workload traces.** The synthetic workloads should be complemented by replaying accounting traces from a production Slurm cluster. A few weeks of records (submit times, allocated nodes, runtimes, requested time limits, and job states) would already serve as a secondary validation of the synthetic results. Real traces also bring realistically inaccurate runtime estimates, which relaxes Assumption 2 for the baselines and P3R alike.

- **Measured power model.** The constant $P_{\text{node}} = 1\,\text{kW}$ per active node (Assumption 7) makes all reported costs relative. Measured active and idle node power, possibly profile-dependent beyond the active-node count, would turn them into euro figures for a concrete machine.

- **Checkpoint overhead and partial checkpointability.** Non-zero checkpoint and restart costs in time and energy (Assumption 5), and workloads in which only a fraction of the jobs is checkpointable (Assumption 4), both shrink the upper bound this thesis estimates. A sensitivity sweep over both dimensions would show how quickly the benefit erodes and which overhead levels still leave price shifting worthwhile.

- **Capacity release during suspension.** Paused jobs currently retain their full reservations (Assumption 6), so every suspension withholds capacity from the queue. Chapter 6 identifies this as probably the assumption with the largest effect on the results, and it is the first one to relax. Releasing the nodes of paused jobs and letting backfilling use them would couple price shifting with utilization instead of trading one against the other. The price is a much harder resume guarantee: a job that gave up its nodes may find them taken when its cheap hour arrives, and the deadline check of Section 4.4.2 no longer suffices. Three variants are worth measuring. First, **release with a resume reservation**, where P3R may only pause a job if it can reserve nodes for the step it plans to run next, in the spirit of EASY's head reservation. Second, **release without a guarantee**, where a paused job returns to the queue and is re-admitted by priority, which is what Slurm's requeue mode does and which trades deadline risk for utilization. Third, **partial release**, where only the nodes the remaining profile does not need are given up, which is possible because the profile is known and is the variant closest to the current model. The comparison would show how much of the utilization loss at medium load is recoverable, at what cost in missed deadlines, and whether pausing can save money even under overload, where it currently cannot.

  Releasing nodes also changes the planning problem, and the current scheduler cannot solve the new one. The dynamic program of Section 4.4.3 plans every job on its own. That is valid only because a started job keeps its nodes, so whichever hours it picks, its capacity is already there. Once paused jobs give their nodes back, a job's plan for a later hour holds only if enough nodes are free in that hour, and that depends on the plans of all other jobs. Deciding which job runs in which hour then becomes one joint problem over all jobs and hours, under the capacity limit and the deadlines. Already its simplest case is hard: deciding whether one-hour jobs of different widths fit into a given number of hours before a common deadline is bin packing, which is NP-complete [7]. Two kinds of planner fit this problem. An **exact planner** writes the next one or two days as a time-indexed integer program, with one decision per job and hour, and solves it again every hour as new jobs and prices arrive. A **heuristic planner** keeps the per-job dynamic program but treats the capacity of future hours as a shared resource. It plans the jobs one after another in priority order, and each plan reserves its nodes in the hours it picks, so that later jobs plan around them. The

exact planner is feasible only for short windows, while the heuristic scales to the whole cluster. Comparing both on the same workloads would show how much of the saving the cheaper planner gives away.

- **Forecast uncertainty.** The known full-year price signal (Assumption 3) should be replaced by day-ahead-only knowledge and then by imperfect forecasts, to measure how robust the savings are to signal quality.

- **Cluster-level energy accounting.** Extending the active-job cost model (Assumption 8) with idle, powered-on, and sleep node states would capture the full electricity bill. It would also expose the interaction between price-aware shifting, temporal job packing, and node power-down decisions, which the current model excludes.

- **Richer policies and baselines.** The hourly run-or-skip rule could be generalized into a scalarized multi-objective decision over execution paths that weights cost, waiting, deadline risk, and profile placement. The comparison could be broadened by conservative backfilling, shortest-job-first, and a non-preemptive price-aware delayed-start policy. The last of these would show how much of the benefit requires pausing at all. An eligibility rule that restricts shifting to jobs above a runtime or wait-tolerance threshold, as Section 6.1 suggests, belongs in the same sweep.

- **Fair-share priorities.** The AIP cluster orders its queue by fair share [17], which the simulator does not model (Assumption 9). Adding users with usage histories to the workload, and combining the fair-share priority with the deadline order of P3R, for example as one more priority factor, would show how much of the saving and of the deadline advantage remains. The baseline would then be EASY with fair share, as Slurm runs it.

- **Slurm integration.** A prototype based on job holds and releases, scheduler-driven preemption, and profile collection from accounting data would test P3R outside the simulator. A first step is a shadow mode that computes P3R's decisions next to the production scheduler without acting on them, so that its plans can be compared with what Slurm actually did. Such a prototype should also run on heterogeneous partitions (Assumption 9).

Bibliography

[1] Kishwar Ahmed, Jason Liu, and Xingfu Wu. "An Energy Efficient Demand-Response Model for High Performance Computing Systems". In: *2017 IEEE 25th International Symposium on Modeling, Analysis, and Simulation of Computer and Telecommunication Systems (MASCOTS)*. IEEE, 2017, pp. 175–186. DOI: 10.1109/MASCOTS.2017.25.

[2] BDEW Bundesverband der Energie- und Wasserwirtschaft. *BDEW-Strompreisanalyse Herbst 2026*. URL: https://www.bdew.de/service/daten-und-grafiken/bdew-strompreisanalyse/ (visited on 10/12/2026).

[3] Bundesnetzagentur. *SMARD – Strommarktdaten*. URL: https://www.smard.de (visited on 08/06/2026).

[4] EPEX SPOT. *Basics of the Power Market*. URL: https://www.epexspot.com/en/basicspowermarket (visited on 05/06/2026).

[5] Dror G. Feitelson, Larry Rudolph, and Uwe Schwiegelshohn. "Parallel Job Scheduling — A Status Report". In: *Job Scheduling Strategies for Parallel Processing*. Ed. by Dror G. Feitelson, Larry Rudolph, and Uwe Schwiegelshohn. Vol. 3277. Lecture Notes in Computer Science. Springer, 2005, pp. 1–16. DOI: 10.1007/11407522_1.

[6] Dror G. Feitelson, Dan Tsafrir, and David Krakov. "Experience with Using the Parallel Workloads Archive". In: *Journal of Parallel and Distributed Computing* 74.10 (2014), pp. 2967–2982.

[7] Michael R. Garey and David S. Johnson. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. New York: W. H. Freeman, 1979.

[8] Gemeinsame Kommission der Prüfungsausschüsse Bachelor und Master Informatik, Freie Universität Berlin. *Handlungsempfehlung für die Nutzung von KI in schriftlichen Arbeiten, Bachelor und Master Informatik*. Nov. 5, 2025. URL: https://www.imp.fu-berlin.de/fbv/pruefungsbuero/Informationen-Anleitungen-FAQ/Informationen-und-Anleitungen/Handreichung-KI-Nutzung-Informatik.pdf (visited on 10/17/2026).

[9] Íñigo Goiri et al. "GreenSlot: Scheduling Energy Consumption in Green Datacenters". In: *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. ACM, 2011, pp. 1–11. DOI: 10.1145/2063384.2063411.

[10] Intel. *Intel Product Specifications: Xeon Gold 6252, Xeon Platinum 8352Y, Xeon 6960P*. URL: https://www.intel.com/content/www/us/en/ark.html (visited on 10/24/2026).

[11] Alexander Kammeyer. "Digital Twin for Energy-Aware High-Performance Computing". PhD thesis. Freie Universität Berlin, 2025.

[12] David A. Lifka. "The ANL/IBM SP Scheduling System". In: *Job Scheduling Strategies for Parallel Processing*. Ed. by Dror G. Feitelson and Larry Rudolph. Vol. 949. Lecture Notes in Computer Science. Springer, 1995, pp. 295–303. DOI: 10.1007/3-540-60153-8_35.

[13] Barry Linnert, César A. F. De Rose, and Hans-Ulrich Heiß. "Toward a Dynamic Allocation Strategy for Deadline-Oriented Resource and Job Management in HPC Systems". In: *Concurrency and Computation: Practice and Experience* 37.1, e8310 (2025). DOI: 10.1002/cpe.8310.

[14] Ahuva W. Mu'alem and Dror G. Feitelson. "Utilization, Predictability, Workloads, and User Runtime Estimates in Scheduling the IBM SP2 with Backfilling". In: *IEEE Transactions on Parallel and Distributed Systems* 12.6 (2001), pp. 529–543.

[15] Pradip Murali and Sathish S. Vadhiyar. "Metascheduling of HPC Jobs in Day-Ahead Electricity Markets". In: *IEEE Transactions on Parallel and Distributed Systems* 29.3 (2018), pp. 614–627.

[16] SchedMD. *Slurm Workload Manager: Gang Scheduling*. URL: https://slurm.schedmd.com/gang_scheduling.html (visited on 10/12/2026).

[17] SchedMD. *Slurm Workload Manager: Multifactor Priority Plugin*. URL: https://slurm.schedmd.com/priority_multifactor.html (visited on 10/22/2026).

[18] SchedMD. *Slurm Workload Manager: Preemption*. URL: https://slurm.schedmd.com/preempt.html (visited on 10/12/2026).

[19] SchedMD. *Slurm Workload Manager: sbatch*. URL: https://slurm.schedmd.com/sbatch.html (visited on 10/17/2026).

[20] SchedMD. *Slurm Workload Manager: Scheduling Configuration Guide*. URL: https://slurm.schedmd.com/sched_config.html (visited on 10/25/2026).

[21] Jörg Schneider et al. "An Efficient Protocol for Reserving Multiple Grid Resources in Advance". In: *Grid and Services Evolution*. Ed. by Ramin Yahyapour, Domenico Talia, and Norbert Meyer. Boston, MA: Springer, 2009, pp. 1–16. DOI: 10.1007/978-0-387-85966-8_14.

[22] Edi Shmueli and Dror G. Feitelson. "Backfilling with Lookahead to Optimize the Packing of Parallel Jobs". In: *Journal of Parallel and Distributed Computing* 65.9 (2005), pp. 1090–1107.

[23] Srividya Srinivasan et al. "Characterization of Backfilling Strategies for Parallel Job Scheduling". In: *Proceedings of the International Conference on Parallel Processing Workshops*. IEEE Computer Society, 2002, pp. 514–519. DOI: 10.1109/ICPPW.2002.1039773.

[24] Philipp Wiesner et al. “Let’s Wait Awhile: How Temporal Workload Shifting Can Reduce Carbon Emissions in the Cloud”. In: *Proceedings of the 22nd International Middleware Conference (Middleware ’21)*. ACM, 2021, pp. 260–272. DOI: 10.1145/3464298.3493399.

[25] Xu Yang et al. “Integrating Dynamic Pricing of Electricity into Energy Aware Scheduling for HPC Systems”. In: *Proceedings of the International Conference on High Performance Computing, Networking, Storage and Analysis (SC ’13)*. ACM, 2013, pp. 1–11. DOI: 10.1145/2503210.2503264.

[26] Andy B. Yoo, Morris A. Jette, and Mark Grondona. “SLURM: Simple Linux Utility for Resource Management”. In: *Job Scheduling Strategies for Parallel Processing*. Ed. by Dror G. Feitelson, Larry Rudolph, and Uwe Schwiegelshohn. Vol. 2862. Lecture Notes in Computer Science. Springer, 2003, pp. 44–60. DOI: 10.1007/10968987_3.

Appendix A

Use of Tools and AI Assistance

This appendix documents and reflects on my use of AI tools for this thesis, following the recommendation of the examination boards for Bachelor and Master Computer Science at Freie Universität Berlin [8].

A.1 Documentation of AI Use

Table A.1 lists the AI tools I used.

Table A.1: *AI tools used in this thesis.*

Tool	Period	Use
Claude Code (Anthropic) with Claude Fable 5, Claude Fable 5.1, and Claude Opus 5.5	Aug.–Oct. 2026	Text, code changes, reviews, experiments
Hermes, an autonomous agent	Sep. 2026	Experiment runs

I chose the topic, which grew out of the concluding chapter of Kammeyer's dissertation [11], framed it around the AIP cluster, decided the model and its assumptions together with my supervisor, and wrote the first version of the simulator in May and June 2026. From August I used Claude Code to make the later changes to the simulator, to add tests and the scripts for the tables and figures, and to run the experiments and compute the statistics. I also used Claude Code as a writing aid. It drafted passages from my outlines, notes, review comments, and instructions, revised my wording, and corrected grammar. I decided what each section says, reviewed every passage, rewrote or removed what did not reflect my own thinking, and wrote and added passages myself where I found them better or necessary. I decided which experiments, analyses, and corrections to adopt. Besides these tools I used Python with pytest and matplotlib, the SMARD price data [3], pdfLaTeX, and Git, with a private repository for the thesis and a public one for the simulator, the data, and the results (Section 5.1).

I also used Claude Code to review code, text, numbers, and citations. These reviews found the errors in Table A.2, which would otherwise have left several numbers and claims of this thesis wrong. The AI-assisted reviews complemented my own review and my supervisor's rather than replacing them.

Table A.2: *Errors found in AI-assisted reviews and their effect.*

Error	Effect if uncorrected
Baselines dropped late jobs, then stopped starting jobs near the end	Distorted comparison of waiting times and deadline misses
Lookahead ignored the order of profile steps in large tables	About 39 % of long jobs affected, bias toward pausing
Cluster-scale estimate used reserved, not active utilization	Saving overstated by up to 0.9 kEUR per year
Result statements and two citations claimed more than their basis	Overstated significance and sources
Deadline advantage over EASY credited to pausing	Wrong attribution of a main result

A.2 Reflection on AI Use

Without these AI tools, some of my simulations would have produced wrong results, and some of the claims in Chapters 5 and 6 would not hold. I would probably not have found these errors on my own, because I had written the code and assumed that it was correct. The errors were also subtle. They did not produce implausible results. No scheduler completed 40 % more jobs than another, and the price saving always stayed in a plausible range. The price saving looked plausible all along, 25.4 % at low load in the first runs and 26.2 % after the corrections, both on the workload used at that time. The error lay in the comparison of service quality. In the first runs P3R's deadline-miss rate seemed to be 11.7 percentage points higher than EASY's at medium load, while the corrected simulator showed that, on the same workload, it was 4.6 points lower. On the final workload, whose job widths follow the limit of the AIP cluster, the comparison changed again (Section 5.5).

Writing with AI was more difficult. The generated text tends to bring in its own tone and its own arguments. Skills, stored instructions that Claude Code follows, made this manageable, but I still had to rewrite or remove much of the AI-written text because it did not reflect the way I think. Even when I explicitly asked it only to rephrase my argument, it added its own opinions and continued my line of thought. Where it did help in writing was in reviewing what I had written, which it did very well, and in checking the formulas and mathematical explanations, so that I could be sure my claims hold. My conclusion is that AI can produce text that seems perfectly logical, but unless it is used carefully, it becomes hard to tell which arguments come from the author and which were generated in addition to them.

Appendix B

Results of the 60-Day Windows

Tables B.1 and B.2 list the means of the two 60-day windows that Section 5.7 refers to, January to February and July to August 2024. The workloads, policies, and seeds are the same in both windows, so the tables differ only in the price trace.

Table B.1: *Winter window, January to February 2024: means over ten seeds with 95 % confidence intervals. Mean resp. is the mean response time. The settings* $\lambda = 0.01$ *and* 0.05 *are omitted.*

Load	Policy	Paid price (EUR/MWh)	Completed (jobs)	QoS viol. (%)	Mean wait (h)	p95 wait (h)	Mean resp. (h)	Suspended (job-h)
low	FCFS	71.4 ± 1.3	228 ± 10	1.4 ± 0.4	0.0 ± 0.0	0 ± 0	22.9 ± 3.0	0 ± 0
	EASY	71.4 ± 1.3	228 ± 10	1.4 ± 0.4	0.0 ± 0.0	0 ± 0	22.9 ± 3.0	0 ± 0
	$\lambda = 0$	58.6 ± 1.2	228 ± 10	2.4 ± 1.6	3.5 ± 0.6	14 ± 3	39.7 ± 4.6	3018 ± 279
	$\lambda = 0.1$	59.5 ± 1.2	228 ± 10	2.3 ± 1.4	2.5 ± 0.5	11 ± 2	37.9 ± 4.5	2847 ± 260
	$\lambda = 0.25$	62.8 ± 1.1	228 ± 10	1.8 ± 0.8	1.4 ± 0.2	7 ± 1	33.6 ± 3.8	2111 ± 231
	$\lambda = 0.5$	67.6 ± 1.1	228 ± 10	1.5 ± 0.4	0.7 ± 0.1	4 ± 1	27.9 ± 3.2	999 ± 117
	$\lambda = 1$	71.2 ± 1.3	228 ± 10	1.4 ± 0.4	0.0 ± 0.0	0 ± 0	23.1 ± 3.0	34 ± 9
medium	FCFS	70.0 ± 1.2	502 ± 18	20.4 ± 10.3	10.3 ± 7.7	51 ± 29	33.1 ± 8.2	0 ± 0
	EASY	70.0 ± 1.2	505 ± 17	13.4 ± 6.7	4.8 ± 3.6	27 ± 19	27.5 ± 4.1	0 ± 0
	$\lambda = 0$	61.9 ± 1.3	503 ± 16	21.2 ± 5.2	11.8 ± 3.3	51 ± 17	42.7 ± 3.1	4315 ± 421
	$\lambda = 0.1$	62.3 ± 1.3	504 ± 16	20.2 ± 5.1	10.4 ± 3.4	47 ± 20	41.2 ± 3.2	4250 ± 373
	$\lambda = 0.25$	64.0 ± 1.1	505 ± 16	16.1 ± 5.1	7.9 ± 3.5	37 ± 19	37.2 ± 3.6	3461 ± 225
	$\lambda = 0.5$	66.9 ± 1.1	505 ± 16	11.3 ± 4.9	4.8 ± 2.5	24 ± 13	31.0 ± 3.1	1874 ± 50
	$\lambda = 1$	69.7 ± 1.2	506 ± 16	6.1 ± 2.3	2.0 ± 1.2	11 ± 7	24.9 ± 1.9	68 ± 12
high	FCFS	69.8 ± 0.3	654 ± 40	83.7 ± 4.1	135.0 ± 35.7	290 ± 66	157.8 ± 36.3	0 ± 0
	EASY	69.8 ± 0.3	698 ± 36	77.6 ± 5.6	94.4 ± 28.1	237 ± 61	116.5 ± 28.6	0 ± 0
	$\lambda = 0$	65.8 ± 0.6	766 ± 18	55.1 ± 8.7	39.2 ± 12.1	172 ± 38	62.5 ± 11.8	3074 ± 505
	$\lambda = 0.1$	65.9 ± 0.5	774 ± 22	50.0 ± 6.1	33.6 ± 9.2	158 ± 43	57.0 ± 9.2	3249 ± 339
	$\lambda = 0.25$	66.7 ± 0.4	782 ± 13	48.1 ± 8.4	30.8 ± 9.6	145 ± 37	53.6 ± 9.4	2649 ± 392
	$\lambda = 0.5$	68.0 ± 0.4	790 ± 12	42.2 ± 8.5	24.5 ± 7.3	122 ± 34	46.3 ± 7.2	1690 ± 297
	$\lambda = 1$	69.7 ± 0.3	798 ± 13	30.8 ± 7.1	17.8 ± 5.9	94 ± 35	38.1 ± 6.1	58 ± 16
mixed	FCFS	69.9 ± 0.5	508 ± 32	85.8 ± 4.1	174.2 ± 25.5	323 ± 60	203.1 ± 26.0	0 ± 0
	EASY	69.8 ± 0.4	554 ± 32	79.9 ± 3.5	107.4 ± 18.9	253 ± 62	135.0 ± 19.4	0 ± 0
	$\lambda = 0$	66.1 ± 0.6	605 ± 21	53.8 ± 4.9	41.2 ± 9.1	209 ± 39	69.9 ± 8.8	2841 ± 320
	$\lambda = 0.1$	66.3 ± 0.7	608 ± 17	52.3 ± 3.8	39.7 ± 7.8	201 ± 37	68.5 ± 7.2	2808 ± 241
	$\lambda = 0.25$	66.9 ± 0.5	608 ± 20	48.9 ± 3.1	34.6 ± 3.6	197 ± 35	63.0 ± 3.3	2437 ± 184
	$\lambda = 0.5$	68.4 ± 0.5	614 ± 19	45.7 ± 6.3	32.2 ± 8.6	164 ± 32	59.1 ± 8.5	1401 ± 157
	$\lambda = 1$	69.7 ± 0.5	618 ± 19	37.0 ± 4.4	23.5 ± 4.9	131 ± 36	49.0 ± 5.1	50 ± 16

Table B.2: *Summer window, July to August 2024: means over ten seeds with 95 % confidence intervals. Mean resp. is the mean response time. The settings* $\lambda = 0.01$ *and* 0.05 *are omitted.*

Load	Policy	Paid price (EUR/MWh)	Completed (jobs)	QoS viol. (%)	Mean wait (h)	p95 wait (h)	Mean resp. (h)	Suspended (job-h)
low	FCFS	74.5 ± 2.2	228 ± 10	1.4 ± 0.4	0.0 ± 0.0	0 ± 0	22.9 ± 3.0	0 ± 0
	EASY	74.5 ± 2.2	228 ± 10	1.4 ± 0.4	0.0 ± 0.0	0 ± 0	22.9 ± 3.0	0 ± 0
	$\lambda = 0$	49.4 ± 2.6	228 ± 10	2.3 ± 1.6	2.9 ± 0.7	13 ± 3	39.7 ± 4.8	3153 ± 302
	$\lambda = 0.1$	52.1 ± 2.6	228 ± 10	2.2 ± 1.3	1.9 ± 0.6	10 ± 2	37.8 ± 4.6	2940 ± 278
	$\lambda = 0.25$	59.0 ± 2.4	228 ± 10	1.6 ± 0.6	1.1 ± 0.2	6 ± 1	33.2 ± 3.9	2106 ± 156
	$\lambda = 0.5$	68.2 ± 2.2	228 ± 10	1.4 ± 0.4	0.5 ± 0.1	3 ± 0	27.1 ± 3.1	844 ± 42
	$\lambda = 1$	74.2 ± 2.2	228 ± 10	1.4 ± 0.4	0.0 ± 0.0	0 ± 0	23.0 ± 3.0	24 ± 6
medium	FCFS	74.3 ± 1.0	502 ± 18	20.4 ± 10.3	10.3 ± 7.7	51 ± 29	33.1 ± 8.2	0 ± 0
	EASY	74.4 ± 1.1	505 ± 17	13.4 ± 6.7	4.8 ± 3.6	27 ± 19	27.5 ± 4.1	0 ± 0
	$\lambda = 0$	60.2 ± 3.3	504 ± 16	22.4 ± 5.0	11.8 ± 3.5	52 ± 21	43.1 ± 3.1	4561 ± 437
	$\lambda = 0.1$	61.6 ± 2.9	504 ± 16	20.6 ± 5.0	10.3 ± 3.4	50 ± 20	41.4 ± 3.2	4437 ± 381
	$\lambda = 0.25$	64.9 ± 2.4	505 ± 16	15.6 ± 4.7	7.6 ± 3.2	36 ± 16	37.1 ± 3.2	3576 ± 233
	$\lambda = 0.5$	70.2 ± 1.3	505 ± 16	8.5 ± 3.4	3.4 ± 1.7	17 ± 9	29.1 ± 2.3	1596 ± 51
	$\lambda = 1$	74.3 ± 1.0	506 ± 16	6.0 ± 2.2	2.0 ± 1.2	10 ± 7	24.7 ± 2.0	56 ± 11
high	FCFS	74.4 ± 0.6	654 ± 40	83.7 ± 4.1	135.0 ± 35.7	290 ± 66	157.8 ± 36.3	0 ± 0
	EASY	74.5 ± 0.7	698 ± 36	77.6 ± 5.6	94.4 ± 28.1	237 ± 61	116.5 ± 28.6	0 ± 0
	$\lambda = 0$	69.5 ± 0.8	768 ± 23	55.8 ± 7.2	40.0 ± 12.4	171 ± 39	63.2 ± 12.1	3091 ± 497
	$\lambda = 0.1$	69.9 ± 0.8	770 ± 22	54.0 ± 7.6	36.9 ± 9.6	175 ± 37	60.4 ± 9.5	3127 ± 473
	$\lambda = 0.25$	71.0 ± 0.7	784 ± 10	47.8 ± 7.7	30.0 ± 9.2	145 ± 39	52.8 ± 8.9	2662 ± 381
	$\lambda = 0.5$	72.7 ± 0.6	790 ± 16	38.6 ± 7.3	22.8 ± 7.2	113 ± 36	44.5 ± 7.2	1493 ± 247
	$\lambda = 1$	74.3 ± 0.4	797 ± 14	32.4 ± 8.7	18.8 ± 6.9	96 ± 37	39.2 ± 7.1	53 ± 14
mixed	FCFS	74.4 ± 0.5	508 ± 32	85.8 ± 4.1	174.2 ± 25.5	323 ± 60	203.1 ± 26.0	0 ± 0
	EASY	74.5 ± 0.5	554 ± 32	79.9 ± 3.5	107.4 ± 18.9	253 ± 62	135.0 ± 19.4	0 ± 0
	$\lambda = 0$	69.8 ± 0.7	608 ± 16	54.1 ± 5.6	43.0 ± 12.9	210 ± 37	71.6 ± 12.4	2842 ± 320
	$\lambda = 0.1$	70.1 ± 0.6	606 ± 21	53.9 ± 4.7	40.5 ± 8.0	198 ± 34	69.3 ± 7.9	2766 ± 306
	$\lambda = 0.25$	71.1 ± 0.6	612 ± 18	46.6 ± 5.0	31.8 ± 4.8	181 ± 42	60.0 ± 4.4	2396 ± 246
	$\lambda = 0.5$	73.0 ± 0.4	615 ± 17	40.4 ± 4.0	26.3 ± 4.9	151 ± 43	53.1 ± 5.1	1177 ± 78
	$\lambda = 1$	74.6 ± 0.4	617 ± 21	36.0 ± 5.2	24.4 ± 7.3	132 ± 41	49.9 ± 7.4	38 ± 12